

Services Web (WS) & Architectures SOA

Partie 1: Technologies de distribution

Chapitre 1: Services Web SOAP.

Chapitre 2: Services Web RESTful.

Partie 2: Architectures SOA

Chapitre 1: Introduction.

Chapitre 2: Caractéristiques SOA.

Chapitre3: Démarche SOA.

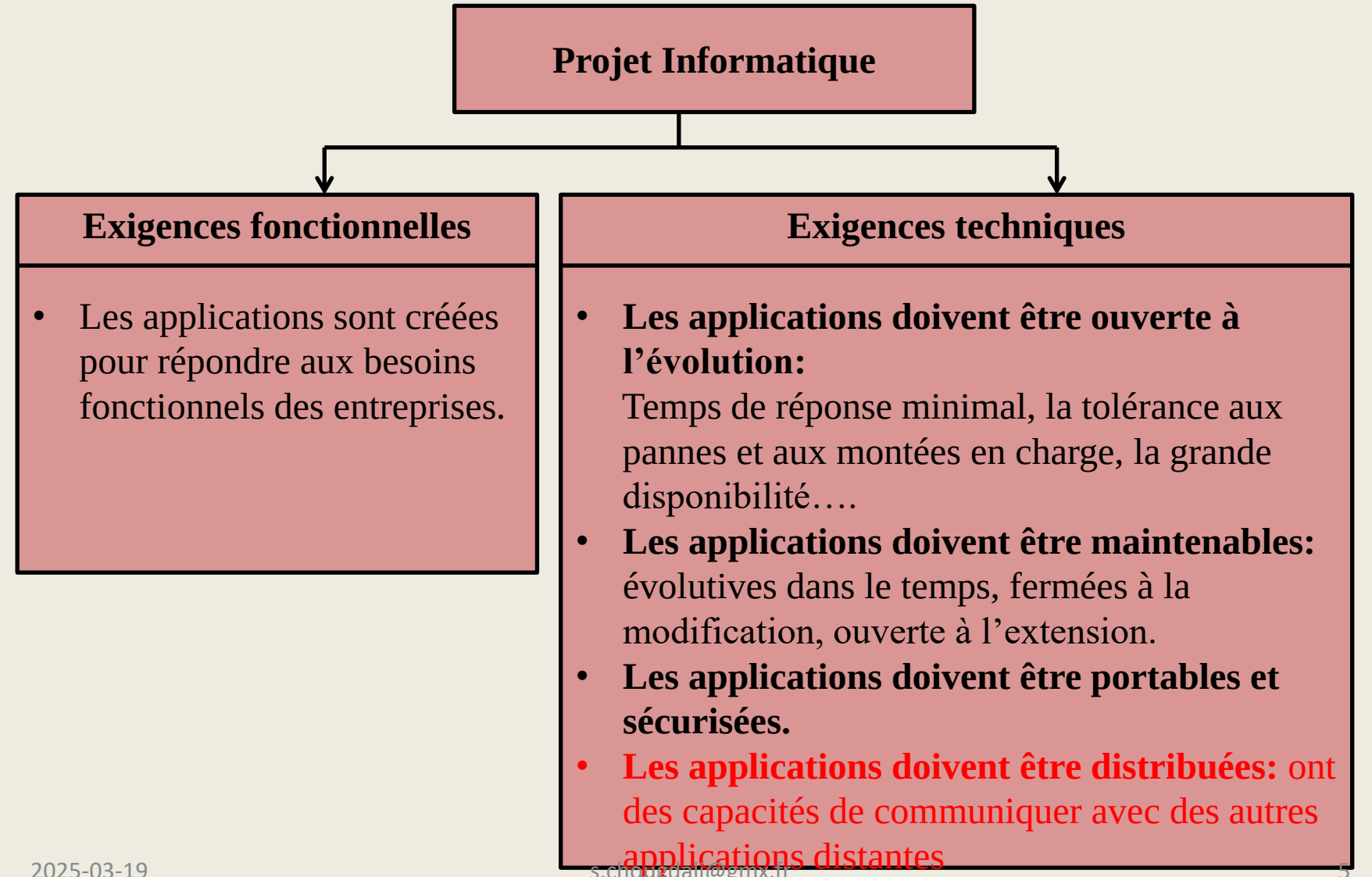
- **La Programmation Orientée Objet (Java).**
- **Le métalangage UML 2.0.**
- **Architecture JEE et Framework Hibernate.**
- **La gouvernance et l'urbanisation des SI.**

Partie 1

Services Web (WS)

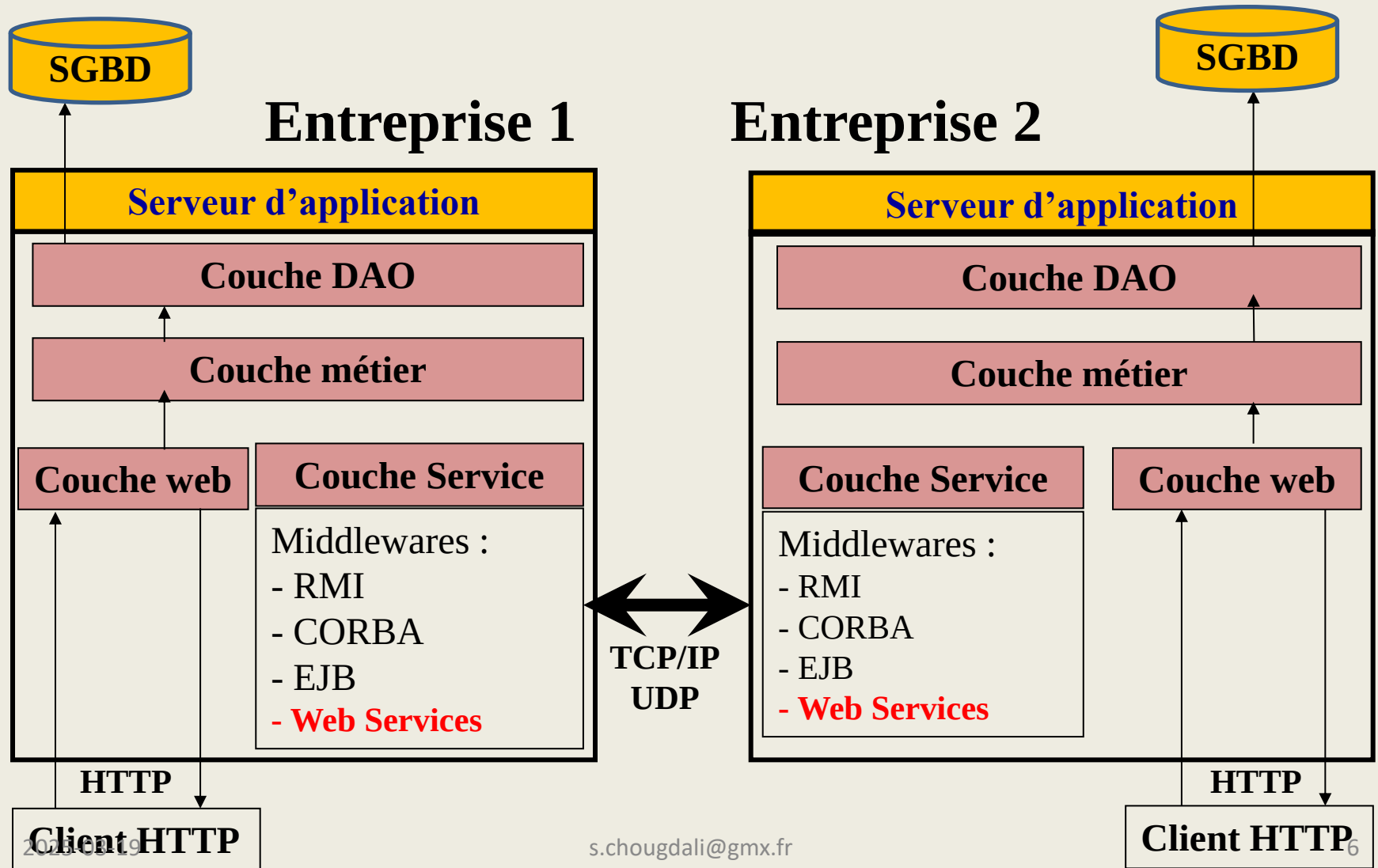
Technologies de distribution

Introduction aux technologies de distribution : Exigences d'un projet informatique.



Partie 1: Technologies de distribution

Introduction aux technologies de distribution: Architecture distribuées

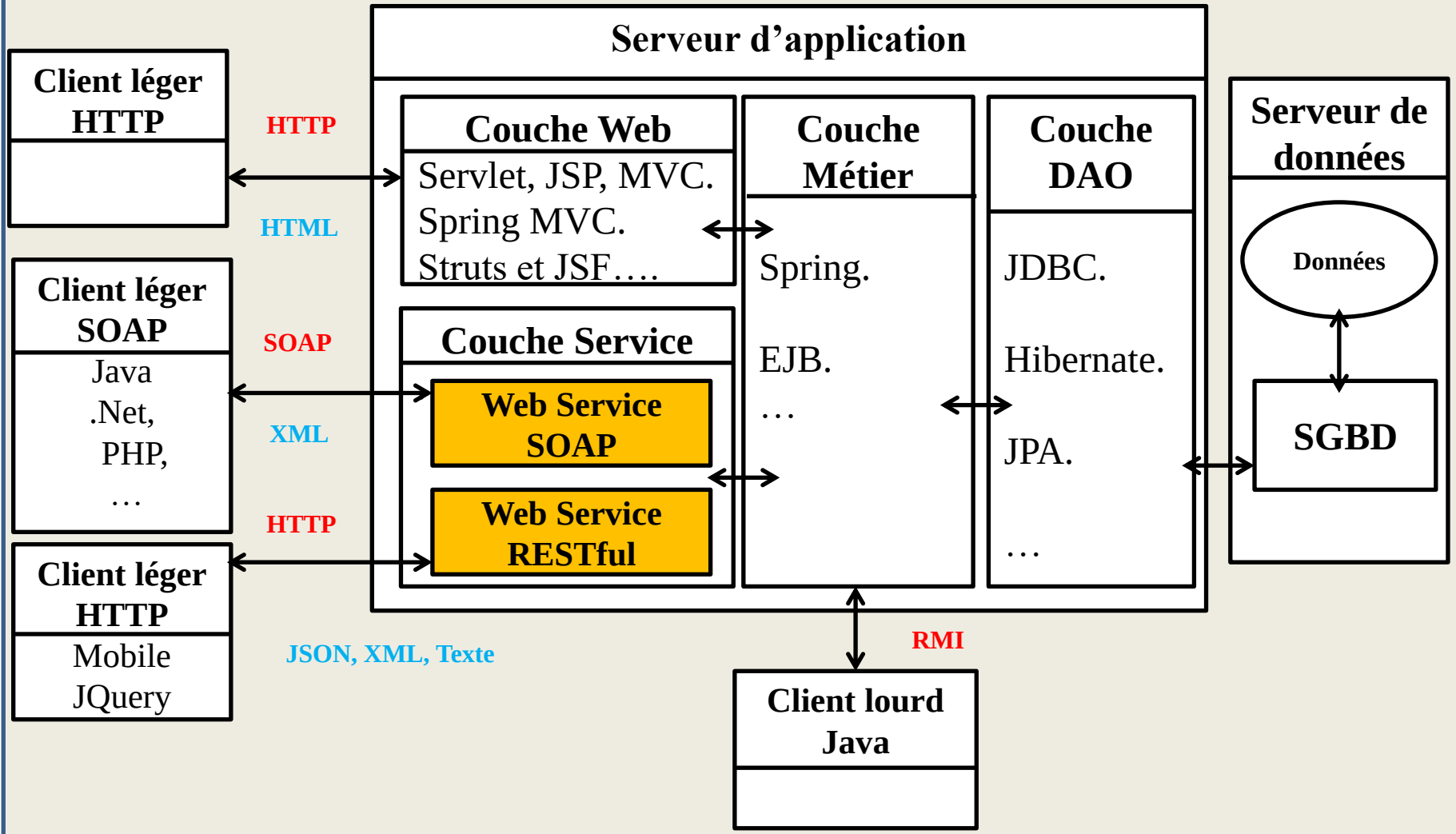


Introduction aux technologies de distribution: Applications distribuées

- Une application distribuée est une application dont les classes sont réparties sur plusieurs machines.
- Une application distribuée permet l'invocation des méthodes à distance.
- Il existe plusieurs technologies pour réaliser des applications distribuées telles que: **RPC** (Remote Procedure Calls), **RMI** (Remote Method Invocation), **CORBA** (Common Object Request Broker Architecture), **EJB** (Entreprise Java Beans), **WS** (Web Services)....
- Toutes les technologies de création des applications distribuées dites aussi collaboratives sont basées sur les **SOCKETS**.

Partie 1: Technologies de distribution.

Introduction aux Web Services.



Introduction aux Web Services.

- Les Web Services sont des applications modulaires qui exécutent des tâches précises et qui respectent un format spécifique.
- Les Web Services à pour rôle principal de faire invoquer des fonctionnalités à distance, et simplifier l'échange de données.
- Les Web Services permettant aux applications de dialoguer à travers le réseau indépendamment de leur plate-forme d'exécution et de leur langage d'implémentation.
- Les Web Services s'inscrivent dans la continuité d'initiatives telles que CORBA (Common Object Request Broker Architecture, de l'OMG) dans le but d'apporter des réponses simples basées sur des technologies et standards reconnus.

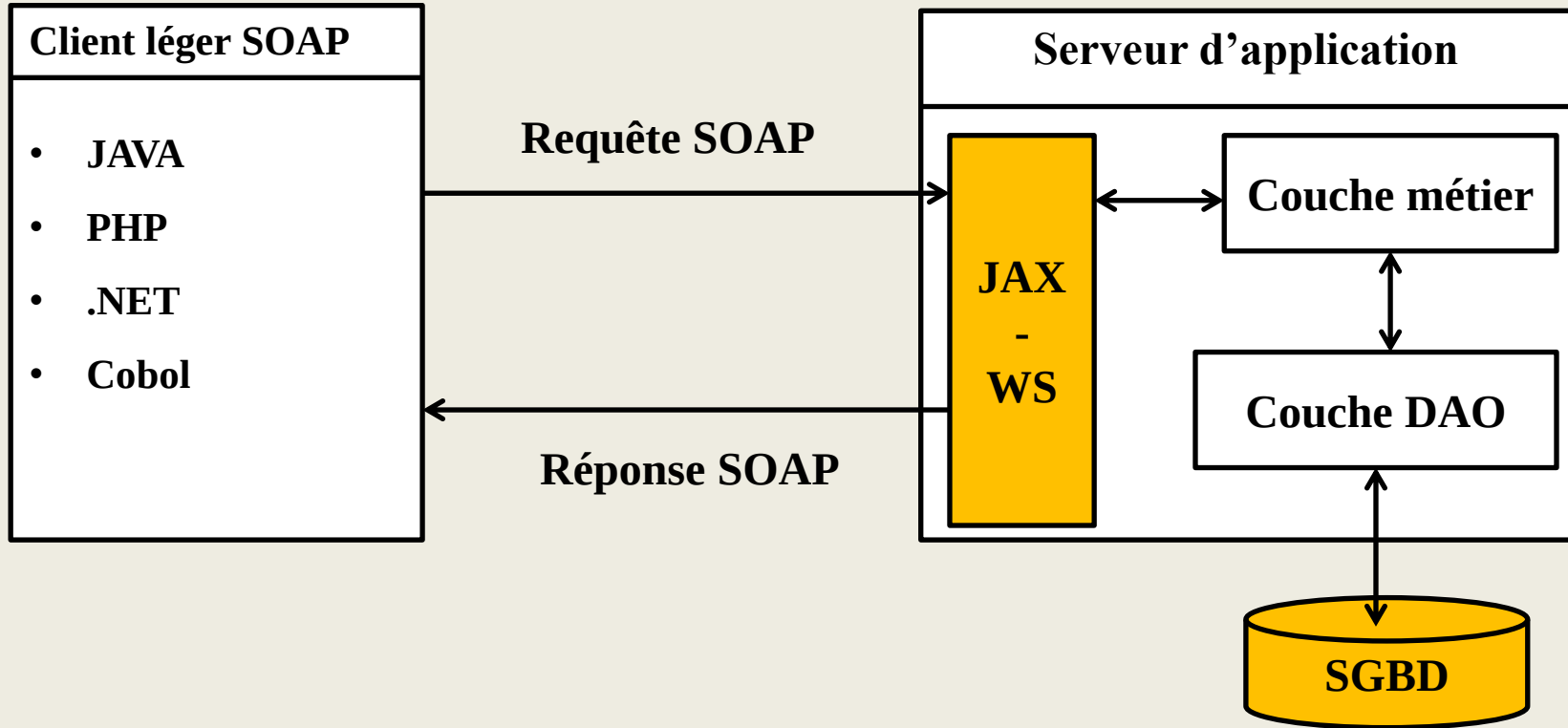
Web Services

SOAP

(JAX-WS)

Introduction au Web Service SOAP : Principe

- Les Web Services SOAP sont basés sur les requêtes et réponse SOAP.
- Les requêtes SOAP sont des requêtes format XML.



Introduction au Web Service SOAP : Format d'une requête SOAP

POST/ Nom_Script SOAP/1.0
REMOTE_ADDR: 212.217.34.01
HTTP_ACCEPT_LANGUAGE: fr
Content_Type: text/xml.
User – Agent: Mozilla/4.0

L'entête d'une requête SOAP
de type POST

--- Saut de ligne---

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <conversion xmlns="http://bktest">
      <montant xmlns="">12</montant>
    </conversion>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Le corps d'une requête SOAP
de type POST (XML)

Introduction au Web Service SOAP : Format d'une réponse SOAP

SOAP/1.0 200 OK
DATE: MON, 05Feb09 14:09:12 GMT
Server: Apache/1.3.24
Mime-Version: 1.0
Last-Modified: Wed06 Oct08
13:02:11GMT
Content_Type: text/xml.

--- Saut de ligne---

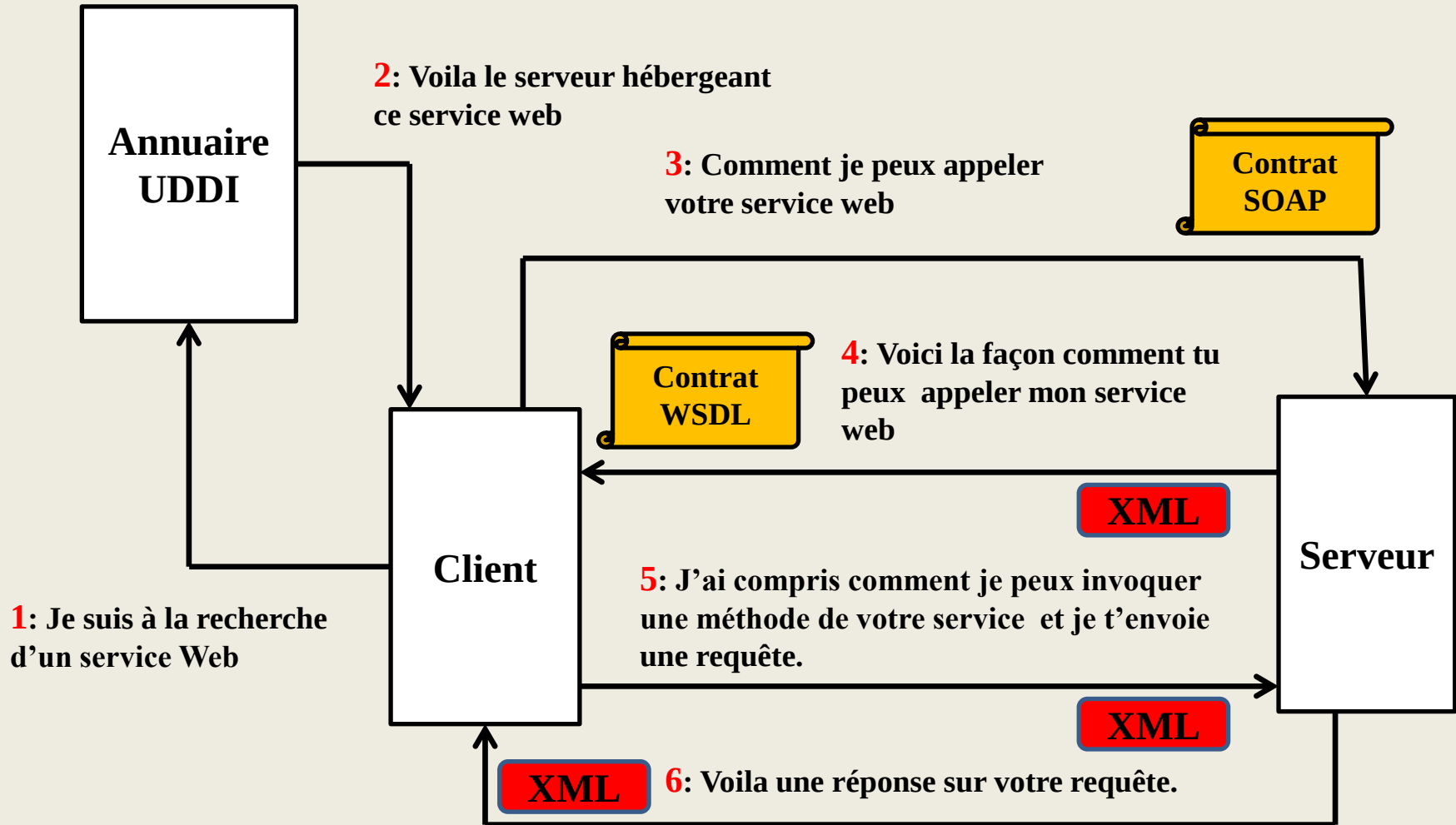
L'entête d'une réponse
SOAP de type POST

```
<?xml version="1.0" ?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:ns1="http://bk/test">
  <soapenv:Body>
    <ns1:conversionResponse>
      <return>132.0</return>
    </ns1:conversionResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

Le corps d'une réponse SOAP
de type POST (XML)

Chapitre 1: Web Services SOAP (JAX-WS).

Architecture de Web Service SOAP



Architecture de Web Service SOAP

L 'architecture des Web Service s'articule sur trois concepts:

- **SOAP (Simple Object Access Protocol):** est un protocole d'échange inter-application indépendant de la plate-forme basée sur la technologie XML.
- **WSDL (Web Services Description Language):** donne la description au format XML des Web Services en précisant les méthodes qui peuvent être invoquées, leurs signatures et le point d'accès (URL, port. ...).
- **UDDI (Universal Description, Discovery and Integration):** normalise une solution d'annuaire distribué de Web Services, permettant à la fois la publication et la recherche de Web Services. Notez bien que l'UDDI se comporte lui-même comme un Web Service dont les méthodes sont appelées via le protocole SOAP.

Cycle de vie d'un Web Service SOAP.

Un Web Service passe par 4 étapes dérivants son cycle de vie:

Etape 1:

- **Déploiement du service Web:** dépend de la plate forme de déploiement par exemple pour Apache on utilise WSSD.

Etape 2:

- **Enregistrement du service Web:** on utilisant WSDL comme langage de description et UDDI comme annuaire.

Etape 3:

- **Découverte du service Web:** le client utilise UDDI et via le protocole SOAP pour découvrir le services Web qu'il veut utiliser.

Etape 4:

- **Invocation du service Web par le client**

Les web Services SOAP avec JAX-WS: Structure

- JAX-WS est la nouvelle appellation de JAX-RPC (Java API for XML Based RPC) qui permet de développer très simplement des services web.
- JAX-WS s'articule sur des annotations, nécessaires pour mapper la correspondance Java-WSDL.
- JAX-WS est une simple classe java qui porte des annotations, permettre par la suite de génère le document WSDL. Ce document est auto-généré par le serveur d'application au moment de déploiement.

Exemple

```
@WebService()  
public class Banque {  
    @WebMethod  
    public double  
    conversionEuroToDh (@WebParam (name="montant") double mt) {  
        ...  
    }  
    ...  
}
```

Les web Services SOAP avec JAX-WS: Principales annotations

Annotation @WebService:

- **@javax.jws.WebService** est l'annotation principale pour dire que cette classe Java présente un web service, alors cette annotation s'applique à une classe.
- **@javax.jws.WebService** comporte plusieurs attributs telles que:
 - **name:** définit le nom du service web, par défaut le service web porte le nom de la classe Java.
 - **serviceName:** définit le nom du service fourni par le service web, par défaut c'est le nom du web service concaténer avec le mot Service.
 - **wsdlLocation:** définit l'URL d'un fichier WSDL, ce fichier est auto-générer lors du déploiement.
 - **endpointInterface:** définit le nom complet de l'interface "endpoint" qui comporte l'ensemble des méthodes du service web.

Les web Services SOAP avec JAX-WS: Principales annotations

Annotation @WebMethod:

- **@javax.jws.WebMethod** est l'annotation qui permet la déclaration d'une méthode accessible par le service web.
- Une méthode qui n'a pas de paramètre de retour peut être annotée par **@Oneway**
- **@javax.jws.WebMethod** peut comporter les attributs suivants:
 - **operationName:** définit le nom de la méthode du service web c'est la valeur de la balise <operation> dans le fichier WSDL, par défaut il prend le nom de la méthode de la classe Java.
 - **exclude:** marque une méthode comme étant non disponible par le service web.

Les web Services SOAP avec JAX-WS: Principales annotations

Annotation @WebParam:

- **@javax.jws.WebParam** permet d'annoter les paramètres d'une méthode du service web.
- **@javax.jws.WebParam** permet aussi de préciser des informations concernant les arguments des opérations définies dans le fichier WSDL.
- **@WebResult** est presque identique, mais elle annote la valeur de retour d'une méthode.
- Les principaux attributs de l'annotation sont:
 - **name:** définit le nom de l'Argument du code Java qui sera utilisé qui sera utilisé dans le fichier WSDL, par défaut elle prend le nom de l'argument Java comme valeur.
 - **header:** indique si la définition du paramètre se trouve dans l'en-tête plutôt que dans le corps, la valeur par défaut étant false c'est-à-dire la définition se trouve.

Les web Services SOAP avec JAX-WS: Le mapping entre WSDL et JAX-WS.

Type Java	Type XML schéma
Byte	xsd:byte
Boolean	xsd:Boolean
Short	xsd:short
Integer	xsd:int
Long	xsd:long
Float	xsd:float
Double	xsd:double
java.lang.String	xsd:string

Les web Services SOAP avec JAX-WS: Le mapping entre WSDL et JAX-WS.

- Le mapping WSDL/JAX-WS utilise les classes enveloppes au lieu des types primitifs.
- On peut utiliser les types primitifs pour les paramètres des méthodes du services web, et l'auto-boxing entre la classe et son type primitif se réalise automatiquement.
- Généralement, la définition de tous les types utilisés par les services web, et les paramètres des méthodes accessibles sont placés dans un fichier qui correspond un schéma XML.
- Alors le fichier WSDL qui porte l'extension (*.wsdl) fait référence à un autre fichier qu'est le schéma XML de l'extension (*.xsd)

Les web Services SOAP avec JAX-WS: La démarche à suivre pour réaliser un JAX-WS.

1. **Développer du service web.**
2. **Générer les Skeletons (artefacts) coté serveur à partir de la classe du Web Service.**
3. **Démarrer le serveur en déployant le web service.**
4. **Générer les Stubs (artefacts) côté client à partir de WSDL.**
5. **Tester les méthodes du Web Services par un client en utilisant les Stubs.**

TP sur les Web Services SOAP – JAX WS. (Déployer sur un projet Java standard)

On reprend le TP gestion des notes, et nous voulons développer un service web SOAP – JAX WS qui donne la possibilité de consulter tous les étudiants et la note d'un étudiant sachant son code.

- Travail demandé:

- 1. Développer et tester la couche DAO on utilisant l'API JDBC.**
- 2. Développer et tester la couche métier.**
- 3. Développer le service web – EtudiantService - .**
- 4. Générer les Skeletons coté serveur à partir de la classe du Web Service.**
- 5. Développer et démarrer le serveur.**

TP sur les Web Services SOAP – JAX WS.

6. **Tester le web service à l'aide d'un client XML.**
7. **Afficher à l'aide d'un navigateur le fichier WSDL et enregistrer ce fichier.**
8. **Générer les Stubs (artefacts) côté client Java à partir de WSDL.**
9. **Développer le client Java et tester les deux méthodes du web services**

Correction du TP.

1- Développement et test de la couche DAO: voir TP JDBC.

2- Développement et test de la couche métier: voir TP JDBC.

3- Développement du service web – EtudiantService -:

```
package ws;

+ import java.util.List;

@WebService(serviceName="etudiantService")

public class EtudiantService {

    private MetierImpl met=new MetierImpl();

    - @WebMethod(operationName="ToutsEtudiant")
      public List<Etudiant> getAllEtudiant(){
          return met.getAllEtudiant();
      }

    - @WebMethod
      public double getAverage(@WebParam(name="codeEtudiant")int codeEtud){
          return met.calculMoyenneEtudiant(codeEtud);
      }

}
```

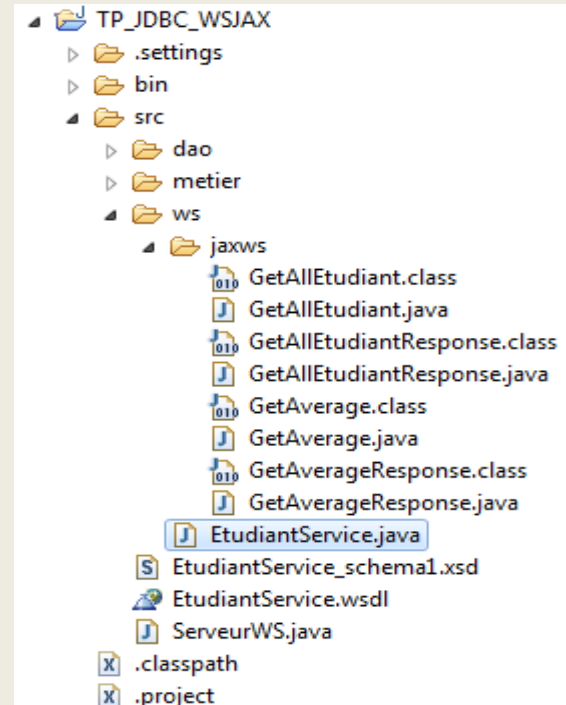
Correction du TP.

4- Générer les Skeletons coté serveur à partir de la classe du Web Service.

On utilisant les lignes des commandes et à l'aide du C?D suivante on génère les Skeletons.

```
C:\Users\Montréal\Desktop\WorkSpaceTPJEE\TP_JDBC_WSJAX\src>wsген -s . -wsdl -cp ../bin ws.EtudiantService
```

- **-s .** Indique au wsген de placer les fichiers source dans le dossier src.
- **-wsdl** est facultatif indique qu'on va créer un fichier wsdl du web services.
- **-cp ../bin** indique à wsген que la classe ws.EtudiantService se trouve dans le dossier bin du projet.



Correction du TP.

5- Développer et démarrer le serveur.

```
import javax.xml.ws.Endpoint;
import ws.EtudiantService;

public class ServeurWS {

    public static void main(String[] args) {
        String url="http://localhost:8585/";
        Endpoint.publish(url, new EtudiantService());
        System.out.println(url);
    }
}
```

NB:

Endpoint.publish permet la publication du web service **EtudiantService()**.

Correction du TP.

6- Tester le web service à l'aide d'un client XML.

- À l'aide de **oXygen XML editor** on peut tester les deux méthodes de notre web service

Requête Fichiers joints

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ns0="http://ws/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns0:getAverage>
      <codeEtudiant>1</codeEtudiant>
    </ns0:getAverage>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Envoyer Configuration de la requête: Ouvrir Enregistrer Régénérer

☐ Ouvrir la réponse dans l'éditeur

Réponse

```
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:getAverageResponse xmlns:ns2="http://ws/">
      <return>16.0</return>
    </ns2:getAverageResponse>
  </S:Body>
```

WSDL

Services: etudiantService

Ports: EtudiantServicePort

Opérations: ToutsEtudiant

Actions

URL: http://localhost:8585/

Action SOAP: Version: ☒ 1.1 ☐ 1.2

Requête Fichiers joints

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:ns0="http://ws/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns0:ToutsEtudiant/>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Envoyer Configuration de la requête: Ouvrir Enregistrer Régénérer

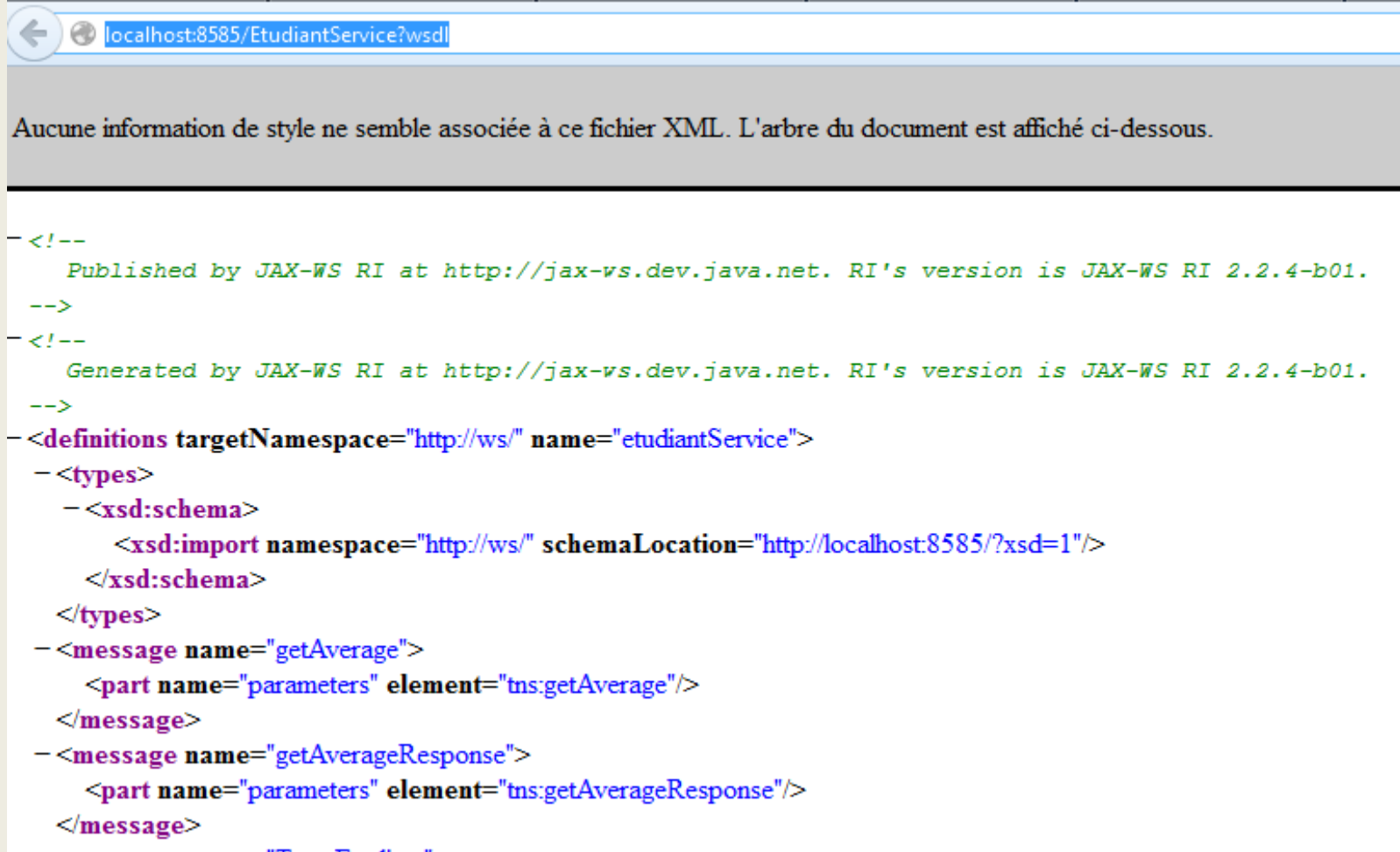
☐ Ouvrir la réponse dans l'éditeur

Réponse

```
<return>
  <adresseEtudiant>CASA</adresseEtudiant>
  <codeEtudiant>13</codeEtudiant>
  <nomEtudiant>MAHRAOUI</nomEtudiant>
  <numtelEtudiant>3456789</numtelEtudiant>
  <prenomEtudiant>ALAA</prenomEtudiant>
```

Correction du TP.

7- Afficher à l'aide d'un navigateur le fichier WSDL et enregistrer ce fichier.



```
<!--  
    Published by JAX-WS RI at http://jax-ws.dev.java.net. RI's version is JAX-WS RI 2.2.4-b01.  
-->  
<!--  
    Generated by JAX-WS RI at http://jax-ws.dev.java.net. RI's version is JAX-WS RI 2.2.4-b01.  
-->  
<definitions targetNamespace="http://ws/" name="etudiantService">  
  <types>  
    <xsd:schema>  
      <xsd:import namespace="http://ws/" schemaLocation="http://localhost:8585/?xsd=1"/>  
    </xsd:schema>  
  </types>  
  <message name="getAverage">  
    <part name="parameters" element="tns:getAverage"/>  
  </message>  
  <message name="getAverageResponse">  
    <part name="parameters" element="tns:getAverageResponse"/>  
  </message>  
</definitions>
```

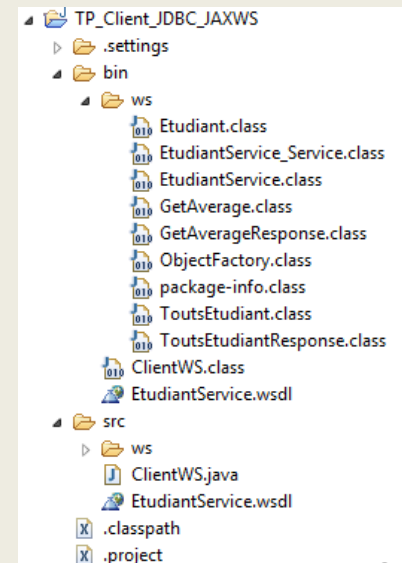
Chapitre 1: Web Services SOAP (JAX-WS).

Correction du TP.

8- Générer les Stubs (artefacts) côté client Java à partir de WSDL .

- Créer un projet java.
- Copier le fichier .wsdl déjà enregistré dans l'étape 6 dans le dossier src. À l'aide de la commande suivante générer les stubs côté client:

```
C:\Users\ASUS\Desktop\WS\WebServiceClient\src>wsimport -keep -p javaclient http://localhost:9898/wstest?wsdl
```



Correction du TP.

9- Développer le client Java et tester les deux méthodes du web services.

```
package javaclient;

public class ClientWSJava {

    public static void main(String[] args) {
        WsTestService stub= new WsTestService();
        WSJaxExemple ws= stub.getWSJaxExemplePort();

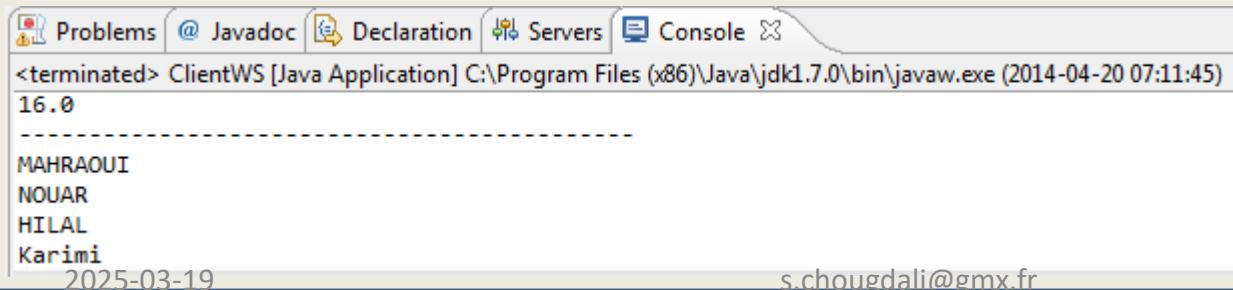
        Etudiant e= ws.consulterUnEtudiant(5);

        System.out.println(e.getNomEtudiant());

        Etudiant et= new Etudiant();
        et.setCodeEtudiant(11);
        et.setNomEtudiant("Karim");
        et.setPrenomEtudiant("Salim");
        et.setNumtelEtudiant("0123456789");
        et.setAdresseEtudiant("Rabat");

        ws.ajouterUtilisateur(et);
    }
}
```

Résultat d'exécution:



```
Problems  Javadoc  Declaration  Servers  Console
<terminated> ClientWS [Java Application] C:\Program Files (x86)\Java\jdk1.7.0\bin\javaw.exe (2014-04-20 07:11:45)
16.0
-----
MAHRAOUI
NOUAR
HILAL
Karimi
2025-03-19
```


Exemple de JAX-WS déployé sur un projet web Dynamic qui utilise Tomcat.

On reprend le même TP – Gestion des notes – mais cette fois ici notre web service JAX-WS sera déployé dans un projet Web dynamic qui utilisera par la suite un serveur d'application qu'est TomCat.

- 1. Créer un projet Web dynamic.**
- 2. Copier la couche DAO et la couche Metier du TP précédent dans le dossier src du projet.**
- 3. Copier le connecteur jdbc sur le dossier lib du projet.**
- 4. Tester la couche métier.**
- 5. Copier la classe EtudiantService du TP précédent qui représente le service web et comporte les deux méthodes getAverage et tousEtudiant dans un package **ws** sur le dossier **src****

Chapitre 1: Web Services SOAP (JAX-WS).

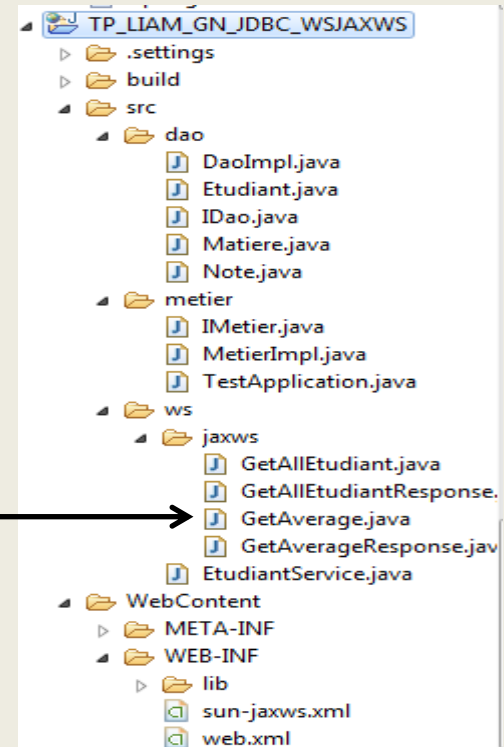
Exemple de JAX-WS déployé sur un projet web Dynamic qui utilise Tomcat.

6. Générer le wsdl du service web par la commande suivante:

```
C:\Users\Montréal\Desktop\WorkspaceDiscoverCasa\TP_LIAM_GN_JDBC_WSJAXWS>wsgen -s src -d build/classes -cp build/classes ws.EtudiantService
```

La structure de notre projet est la suivante:

Vérifier la création des classes de notre services web



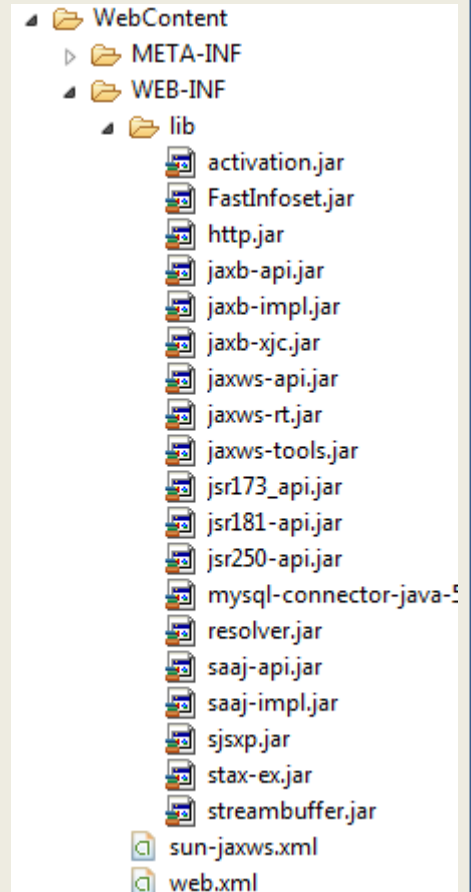
Chapitre 1: Web Services SOAP (JAX-WS).

Exemple de JAX-WS déployé sur un projet web Dynamic qui utilise Tomcat.

7. Copier jaxws jars sur le dossier lib du projet.

8. Déployer le web service via le fichier web.xml:

```
<display-name>TP_LIAM_GN_JDBC_WSJAXWS</display-name>
<listener>
  <listener-class>com.sun.xml.ws.transport.http.servlet.WSServletContextListener</listener-class>
</listener>
<servlet>
  <servlet-name>gn</servlet-name>
  <servlet-class>com.sun.xml.ws.transport.http.servlet.WSServlet </servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>gn</servlet-name>
  <url-pattern>/EtudiantService</url-pattern>
</servlet-mapping>
</web-app>
```



Chapitre 1: Web Services SOAP (JAX-WS).

Exemple de JAX-WS déployé sur un projet web Dynamic qui utilise Tomcat.

9. Créer un fichier sun-jaxws.xml pour la définition de l'Endpoint du service web:

```
<?xml version="1.0" encoding="UTF-8"?>
<endpoints xmlns="http://java.sun.com/xml/ns/jax-ws/ri/runtime" version="2.0">
  <endpoint
    name="gn"
    implementation="ws.EtudiantService"
    url-pattern="/EtudiantService"/>
</endpoints>
```

10. Tester le service web via un Client XML:

Requête Fichiers joints

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:ns0="http://ws/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns0:getAverage>
      <codeEtudiant>1</codeEtudiant>
    </ns0:getAverage>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Envoyer Configuration de la requête: Ouvrir Enregistrer Régénérer

☐ Ouvrir la réponse dans l'éditeur

Réponse

```
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:getAverageResponse xmlns:ns2="http://ws/">
      <return>16.0</return>
    </ns2:getAverageResponse>
  </S:Body>
```

WSDL

Services: etudiantService

Ports: EtudiantServicePort

Opérations: ToutsEtudiant

Actions

URL: http://localhost:8585/

Action SOAP: Version: ☒ 1.1 ☐ 1.2

Requête Fichiers joints

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:ns0="http://ws/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns0:ToutsEtudiant/>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Envoyer Configuration de la requête: Ouvrir Enregistrer Régénérer

☐ Ouvrir la réponse dans l'éditeur

Réponse

```
<return>
  <adresseEtudiant>CASA</adresseEtudiant>
  <codeEtudiant>13</codeEtudiant>
  <nomEtudiant>MAHRAOUI</nomEtudiant>
  <numtelEtudiant>3456789</numtelEtudiant>
  <prenomEtudiant>ALAA</prenomEtudiant>
```

Client Android d'un web services JAX-WS:

1. **Créer un projet Android.**
2. **Copier Ksoap-android-assembly-3.0.0 sur le dossier libs.**
3. **Créer l'interface Homme-Machine (Layout).**

Client Android d'un web services JAX-WS:

4. Programmer Le MainActivity: exemple de code pour la méthode getAverage du services web.

```
btn_ok.setOnClickListener(new OnClickListener() {

    @Override
    public void onClick(View arg0) {

        final String NAMESPACE = "http://ws/";
        final String URL="http://192.168.1.9:8080/TP_LIAM_GN_JDBC_WSJAXWS/EtudiantService?WSDL";
        final String METHOD_NAME="getAverage";
        final String SOAP_ACTION="http://ws/getAverage";
        String s=code_etud.getText().toString();
        System.out.println(s);
        int code=Integer.parseInt(s);
        System.out.println(code);
        SoapObject request=new SoapObject(NAMESPACE, METHOD_NAME);
        request.addProperty("codeEtudiant", code);
        SoapSerializationEnvelope envelope=new SoapSerializationEnvelope(SoapEnvelope.VER11);
        envelope.setOutputSoapObject(request);
        HttpTransportSE androidHttpTransport=new HttpTransportSE(URL);
        try {
            androidHttpTransport.call(SOAP_ACTION, envelope);
            //SoapPrimitive resultsRequestSOAP=(SoapPrimitive) envelope.getResponse();
            //moy.setText(resultsRequestSOAP.toString());
            SoapObject reponseSoap=(SoapObject) envelope.bodyIn;
            moy.setText(reponseSoap.getProperty(0).toString());
        } catch (Exception e) {
            e.printStackTrace();
        }

    }

});
```

5. Tester l'application sur un device Android ou un Émulateur distant.

Web Services

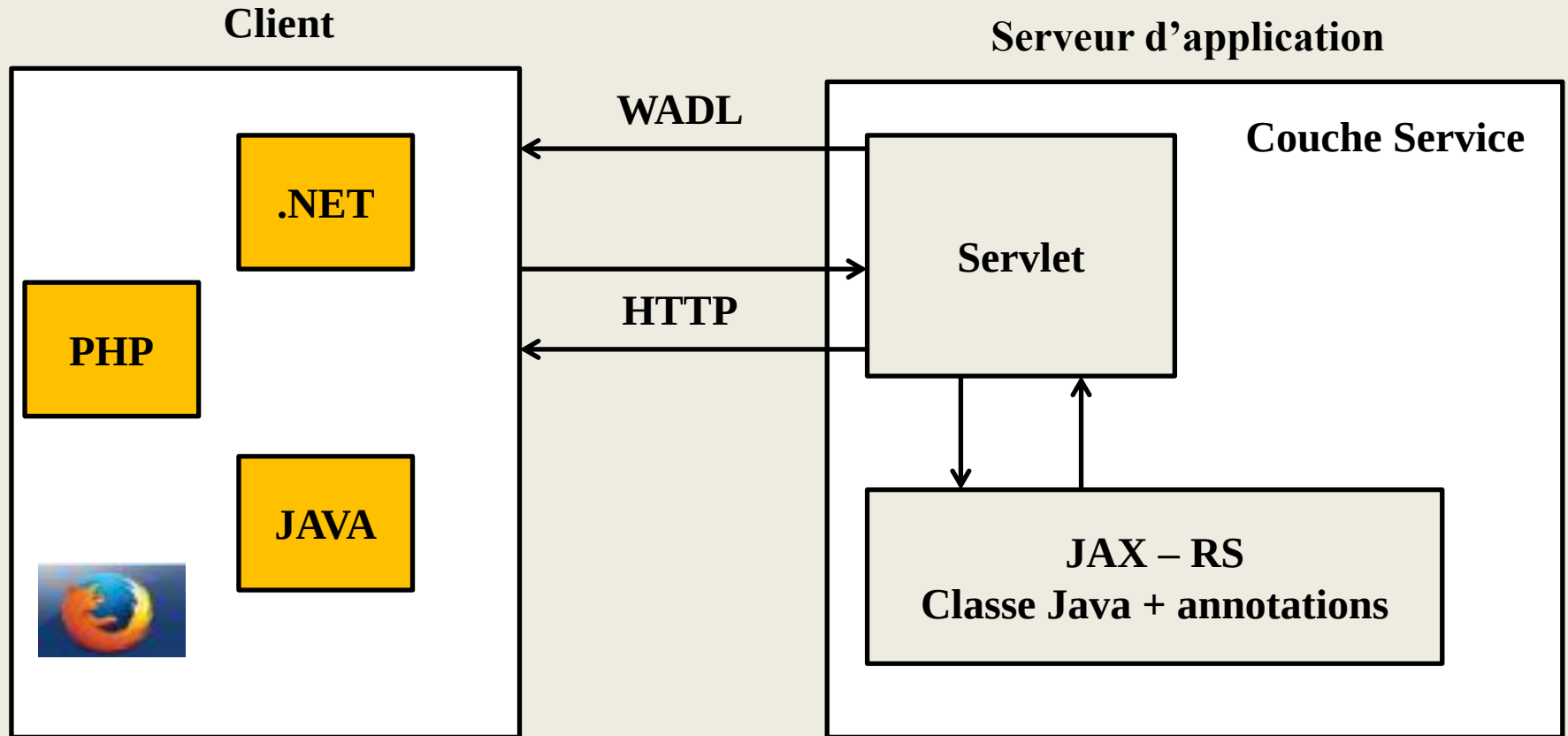
RESTful

JAX - RS

Web Services RESTful – JAX RS: Introduction.

- **JAX – RS est l’acronyme de Java API for RESTful Web Services.**
- **Depuis la version 1.1, JAX – RS fait partie intégrante des spécifications Java EE 6.**
- **Cette spécification décrit uniquement la mise en œuvre de services web REST coté serveur.**
- **Le développement des services web s’articule sur l’utilisation des classes Java et des annotations.**
- **Un service web est déployé dans une application web, pas de fichiers de configuration.**
- **Le développement des services web RESTful JAX-RS est basé sur des POJO (Plain Old Java Object) en utilisant des annotations spécifiques à JAX-RS.**

Web Services RESTful – JAX RS: Architecture.



- **HTTP:** Utilisation du web service via des requêtes et réponses HTTP.
- **WADL:** descripteur du service web nécessaire pour générer le client.

Web Services RESTful – JAX RS: Exemple simple.

On développe un exemple de service web RESTful – JAX RS on utilisant le Framework RestEasy du Jboss, qui comporte une seule méthode qui permet d'afficher le message suivant: **Bonjour mes étudiants de la licence IAM.**

Les outils nécessaires:

- Un IDE: Eclipse.
- Un serveur JEE: JBoss7.1.

1- Créer un projet web dynamic.

2- Développer le web service RESTful – JAX RS à l'aide des annotations.

3- Déployer la servlet et créer un listener sur le fichier web.xml.

4- Tester le service web à l'aide d'un navigateur.

Web Services RESTful – JAX RS: Exemple simple.

Correction:

2- Le web service RESTful – JAX RS est une classe Java avec des annotations:

@ Path: Définition d'un chemin de ressource
pour associer une ressource bonjour à un e URI

```
import javax.ws.rs.GET;  
import javax.ws.rs.Path;  
import javax.ws.rs.Produces;  
import javax.ws.rs.core.MediaType;
```

```
@Path("/bonjour")  
public class TestRESTful {  
  
    @GET  
    @Produces(MediaType.APPLICATION_JSON)  
    public String salut(){  
        return "Bonjour mes etudiants de la LIAM";  
    }  
}
```

@ Produces: nécessaire pour la définition
du type MIME de la réponse ici format
JSON

@ GET: Lecture de la ressource via la une
requête HTTP de type GET

Web Services RESTful – JAX RS: Exemple simple.

Correction:

3- Déploiement de la servlet et la définition d'un listener sur le fichier Web.xml :

```
<web-app>
  <display-name>Archetype Created Web Application</display-name>
  <context-param>
    <param-name>resteasy.scan</param-name>
    <param-value>true</param-value>
  </context-param>

  <listener>
    <listener-class>org.jboss.resteasy.plugins.server.servlet.ResteasyBootstrap</listener-class>
  </listener>

  <servlet>
    <servlet-name>Resteasy</servlet-name>
    <servlet-class>org.jboss.resteasy.plugins.server.servlet.HttpServletDispatcher</servlet-class>
  </servlet>

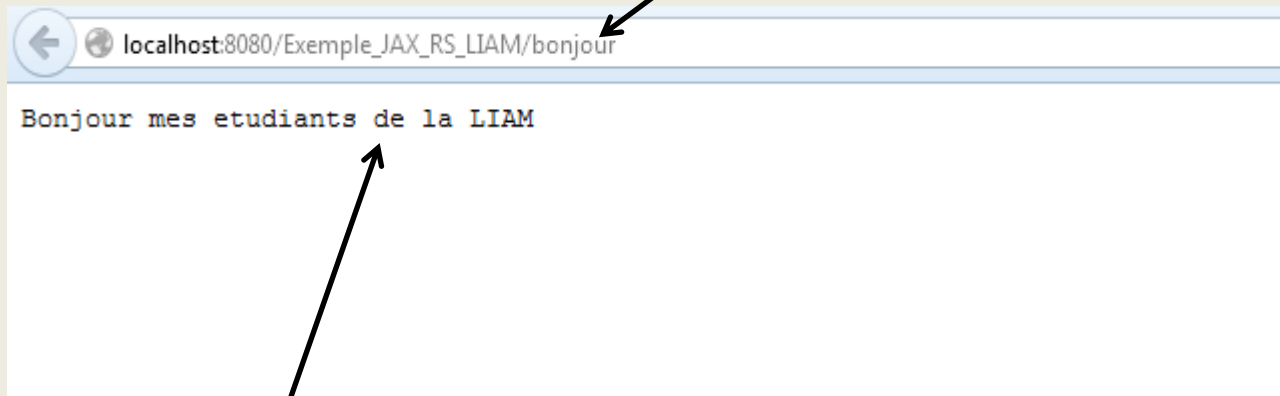
  <servlet-mapping>
    <servlet-name>Resteasy</servlet-name>
    <url-pattern>/*</url-pattern>
  </servlet-mapping>
</web-app>
```

Web Services RESTful – JAX RS: Exemple simple.

Correction:

4- Tester le service web par un navigateur:

**L'envoi d'une requête HTTP de type
GET pour accéder à la ressource
bonjour se fait via un URI**

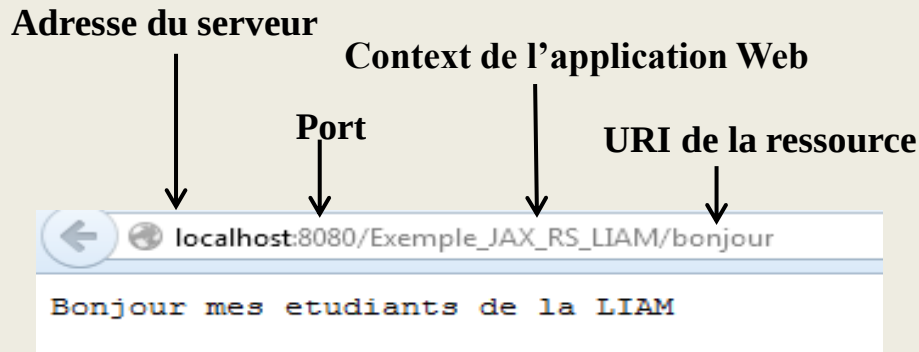


**La réponse est directement affichée par
le navigateur puisque qu'il s'agit d'un type
MIME reconnu**

Web Services RESTful – JAX RS: Principales annotations.

- **@Path**: définit des ressources des ressources appelées racines (Root Resource Class).

Exemple :

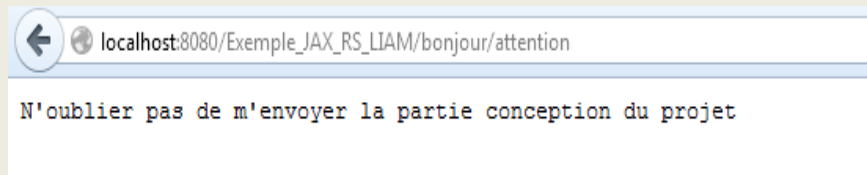


```
import javax.ws.rs.GET;

@Path("/bonjour")
public class TestRESTful {

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public String salut(){
        return "Bonjour mes etudiants de la LIAM";
    }
}
```

NB: l'annotation **@Path** peut annoter des méthodes de la classe web services.



```
import javax.ws.rs.GET;

@Path("/bonjour")
public class TestRESTful {

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public String salut(){
        return "Bonjour mes etudiants de la LIAM";
    }

    @GET
    @Path("/attention")
    @Produces("text/plain")
    public String salut2()
    {
        return "N'oublier pas de m'envoyer la partie conception du projet";
    }
}
```

Web Services RESTful – JAX RS: Principales annotations.

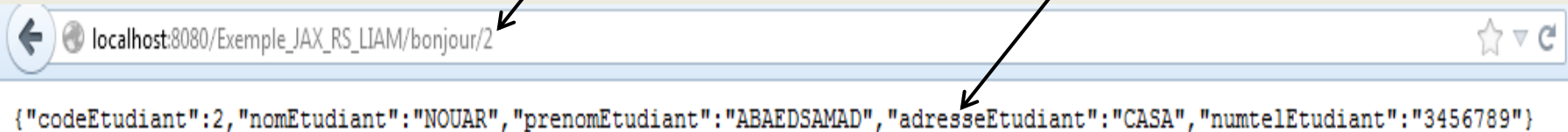
- **@Path:** permet de définir des expressions plus complexe appelées Template Paramètres.

Exemple:

```
@GET
@Path("/{id}")
@Produces(MediaType.APPLICATION_JSON)
public Etudiant getEtudiant(@PathParam("id")int code){
    return met.getEtudiant(code);
}
```

Code étudiant

La requête retourne l'étudiant qu'a le code 2 en format JSON



The screenshot shows a web browser window with the address bar containing the URL `localhost:8080/Exemple_JAX_RS_LIAM/bonjour/2`. Below the address bar, the JSON response is displayed: `{"codeEtudiant":2,"nomEtudiant":"NOUAR","prenomEtudiant":"ABAEDSAMAD","adresseEtudiant":"CASA","numtelEtudiant":"3456789"}`. Two arrows point from the text annotations above to the browser window: one points to the `2` in the URL path, and the other points to the `"codeEtudiant":2` in the JSON response.

- Une sub-ressource locator est une méthode qui doit être annoter par **@Path** toute seule et n'est annoté par GET, POST,..... A pour intérêt de pouvoir rediriger vers une autre classe source

Web Services RESTful – JAX RS: Principales annotations.

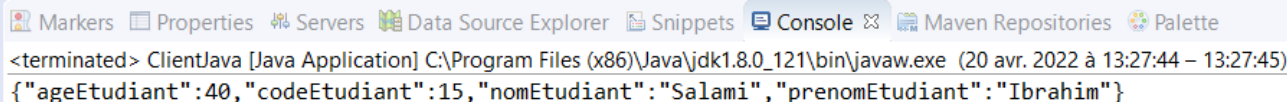
- **@GET, @POST, @PUT, @DELETE**: des annotations offertes par JAX-RS pour définir des requêtes HTTP.
- Ces annotations ne sont pas utilisées que sur des méthodes Java de la classe source.
- Les spécifications JAX-RS n'impose pas de respecter les conventions définie par le style REST, par exemple on peut supprimer une ressource on utilisant GET à la place de DELETE.
- **@PathParam, @FormParam, @HeaderParam,...** : permettant d'extraire les paramètre de la requête.
- **@Produces**: utilisée pour définir le type MIME produit par la méthode.

Web Services RESTful – JAX RS: Client Java.

1. Créer un projet Java,
2. Importer le package jar du Framework jersey,
3. Exemple du code client Java du JAX-RS:

```
9 public class ClientJava {
10
11     public static void main(String[] args) {
12
13         // Client cl= ClientBuilder.newClient();
14         // String msg= cl.target("http://localhost:8080/tptestrestsrddi/myresource").request().get(String.class);
15         // System.out.println(msg);
16
17         Client cl= ClientBuilder.newClient();
18         String entity= cl.target("http://localhost:8080/tptestrestsrddi/myresource/etudiant/15")
19             .request(MediaType.APPLICATION_JSON).header("Head", true).get(String.class);
20         System.out.println(entity);
21
22
23
24     }
25
26 }
```

4. Résultat de l'exécution:



The screenshot shows the IDE's console window with the following output:

```
<terminated> ClientJava [Java Application] C:\Program Files (x86)\Java\jdk1.8.0_121\bin\javaw.exe (20 avr. 2022 à 13:27:44 – 13:27:45)
{"ageEtudiant":40,"codeEtudiant":15,"nomEtudiant":"Salami","prenomEtudiant":"Ibrahim"}
```

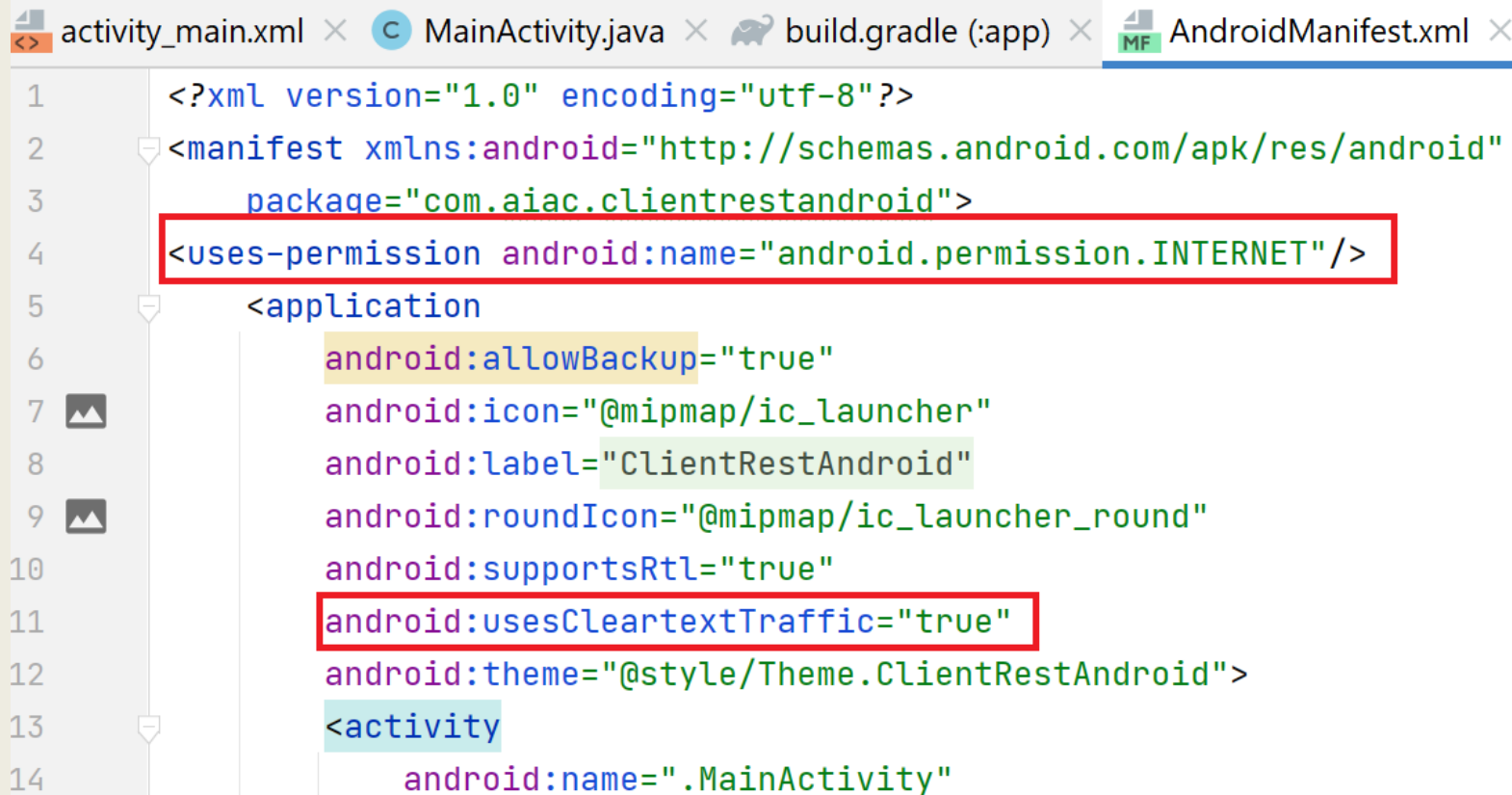
Web Services RESTful – JAX RS: Client Android.

1. Créer un projet Android,
2. Ajouter une dépendance de l'outil volley comme parseur un fichier Json,

```
dependencies {  
  
    implementation 'androidx.appcompat:appcompat:1.4.0'  
    implementation 'com.google.android.material:material:1.4.0'  
    implementation 'androidx.constraintlayout:constraintlayout:2.1.2'  
    testImplementation 'junit:junit:4.+'  
    androidTestImplementation 'androidx.test.ext:junit:1.1.3'  
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.4.0'  
    implementation 'com.android.volley:volley:1.1.0'  
}
```

Web Services RESTful – JAX RS: Client Android.

3. Configurer le projet



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     package="com.aiac.clientrestandroid">
4     <uses-permission android:name="android.permission.INTERNET"/>
5     <application
6         android:allowBackup="true"
7         android:icon="@mipmap/ic_launcher"
8         android:label="ClientRestAndroid"
9         android:roundIcon="@mipmap/ic_launcher_round"
10        android:supportsRtl="true"
11        android:usesCleartextTraffic="true"
12        android:theme="@style/Theme.ClientRestAndroid">
13         <activity
14             android:name=".MainActivity"
```

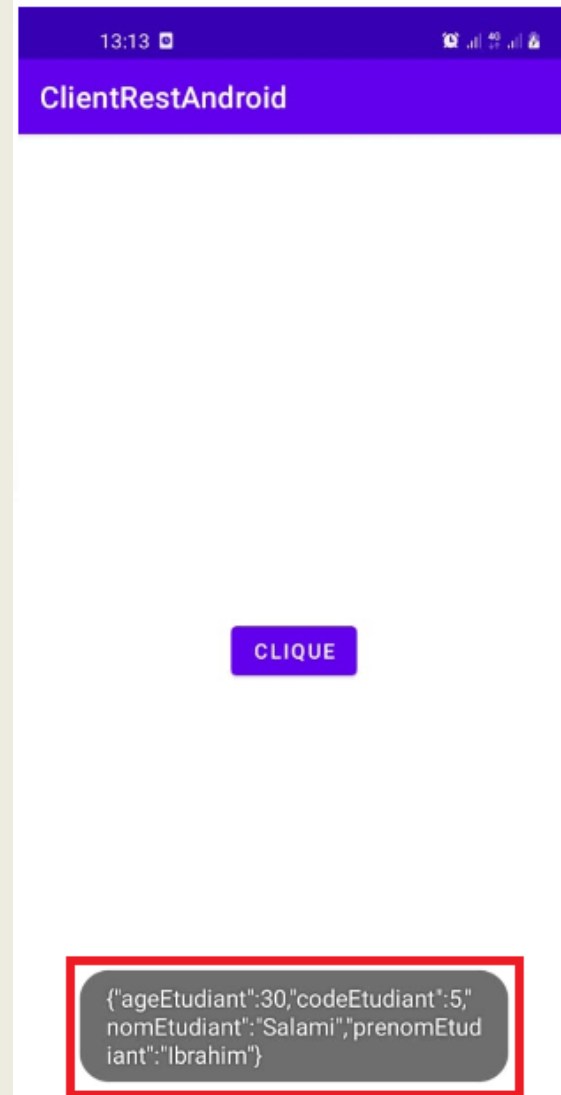
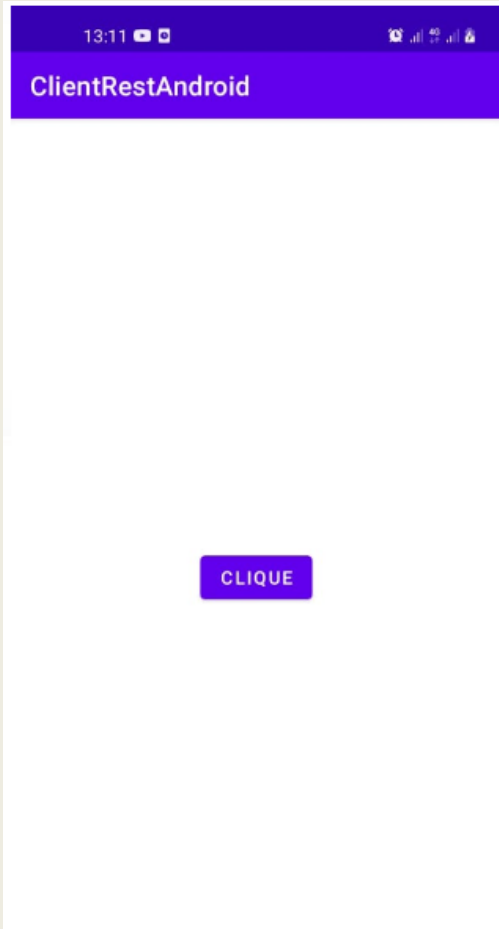
Web Services RESTful – JAX RS: Client Android.

4. Exemple de code:

```
public class MainActivity extends AppCompatActivity {
    Button b;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        RequestQueue reqQ= Volley.newRequestQueue( context: this);
        final String URL= "http://192.168.100.181:8080/tptestrestsrdi/myresource/etudiant/5";
        b= findViewById(R.id.btn);
        b.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                JsonObjectRequest getRequest = new JsonObjectRequest(Request.Method.GET, URL,
                    jsonRequest: null,
                    new Response.Listener<JSONObject>(){
                        @Override
                        public void onResponse(JSONObject response) {
                            Toast.makeText(getApplicationContext(),
                                response.toString(), Toast.LENGTH_SHORT).
                                show();}},
                    new Response.ErrorListener(){
                        @Override
                        public void onErrorResponse(VolleyError error) {
                            Log.d( tag: "Error.Response", error.toString());}});
                reqQ.add(getRequest);
            }
        });
    }
}
```

Web Services RESTful – JAX RS: Client Android.

Exécution:



Web Services RESTful – JAX RS: TP

Nous proposons la réalisation d'une application qui gère les comptes bancaires. Un client est identifié par son code auto incrémente, nom, prénom et un numéro de téléphone, il peut avoir un ou plusieurs comptes, un compte est caractérisé par un numéro de compte alphanumérique, une date de création et un solde. Il existe deux types de comptes compte épargne qui possède un taux d'intérêt et un compte courant qui offre une découverte. Un compte peut avoir une ou plusieurs opérations et chaque opération est définie par un code auto incrémente, une désignation, et un montant de l'opération.

- L'application doit être capable à répondre sur les requêtes suivantes:
- Ajouter un client, un compte et une opération.
- Consulter un client sachant son code. Consulter tous les clients.
- Supprimer un client sachant son code ...

Web Services RESTful – JAX RS: TP

1- La phase conceptuelle (30min):

- Donner un schéma du projet.
- Donner le MCD et MLD.
- Donner le diagramme des cas d'utilisation et diagramme de classes.

2- Implémentation (1h30min):

- Créer une base de données DB_GestionCompte, la création des tables n'est pas demandée, elles seront créer d'une façon automatique par Hibernate.
- Créer la couche Dao en utilisant le Framework Hibernate.
- Créer la couche métier.

Web Services RESTful – JAX RS: TP

3- Implémentation d'un service web (1h30):

- Implémenter un web services type RESTful – JAX RS qui comporte l'ensemble des méthodes.
- Déployer le projet.
- Tester les méthodes par un navigateur web.

Web Services RESTful – JAX RS: TP

Nous souhaitons concevoir et implémenter un Système d'informations qui permet de gérer un cinéma.

Le cinéma contient un ensemble de salles. Chaque salle est définie par un numéro, un nom et le nombre de places qu'elle offre. Il existe deux types de salles les salles normales et les salles VIP. Le prix d'une place est défini en fonction du type de salle. Nous supposons que le prix d'une place dans une salle normale est fixé à 30 DH alors que le prix d'une place dans une salle VIP est fixé à 60 DH. Le cinéma mémorise également un ensemble de films. Chaque film est défini par son titre et son réalisateur. Le cinéma mémorise également les séances de projections. Chaque séance de projection concerne un film et se déroule dans une salle. Pour chaque séance de projection, on doit mémoriser la date de projection ainsi que le nombre de places vendues. Nous prévoyons également de définir dans la classe Séance une méthode qui permet de vendre un certain nombre de places.

Cette application peut être exploitée par un utilisateur, un vendeur et l'administrateur.



Partie 2

Architecture SOA

Définitions:

Architecture ?



L'art de concevoir et de construction des édifices !!

Ou

La structure et l'organisation de quelque chose, d'un ensemble des choses !!

Définitions:

Architecture en informatique ?



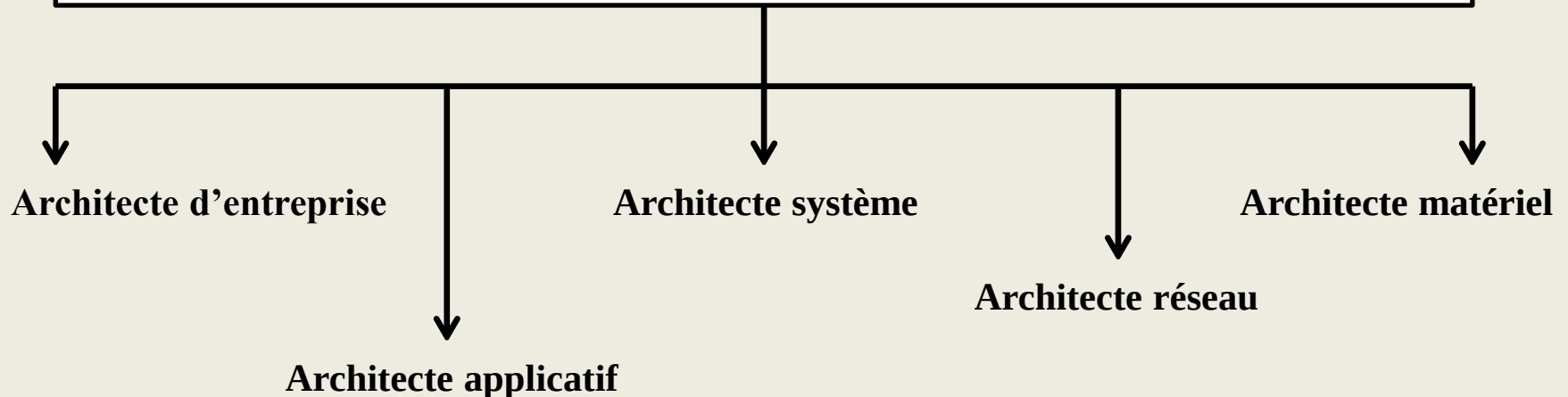
Une description formelle d'un système, la définition de son objet, les fonctions, les propriétés visibles de l'extérieur, et des interfaces. Il inclut également la description des composants internes du système et leurs relations, ainsi que les principes de la conception, de l'exploitation et de l'évolution du système. (Srinivasan L., 2005)

Définitions:

Architecte en informatique ?



La personne chargée de l'analyse technique nécessaire à la conception du *diagramme d'architecture*, c'est-à-dire le plan de construction d'un logiciel, d'un réseau, d'une base de données



Définitions:

Service ?



Fonction de quelqu'un qui sert une cause ou qui aide une personne.

Synonyme travail.

**branche d'activité d'une entreprise. Ex Le service marketing.
Synonyme administration**

Définitions:

SOA ?



Architecture Orientée Services

- **Un style de construction fiable des systèmes distribués.**
- **Faiblesse de l'association entre les services (Couplage faible).**

Mettre en place d'une Architecture Orientée Services

=

Modéliser le système informatique en se focalisant sur les services métier de l'entreprise

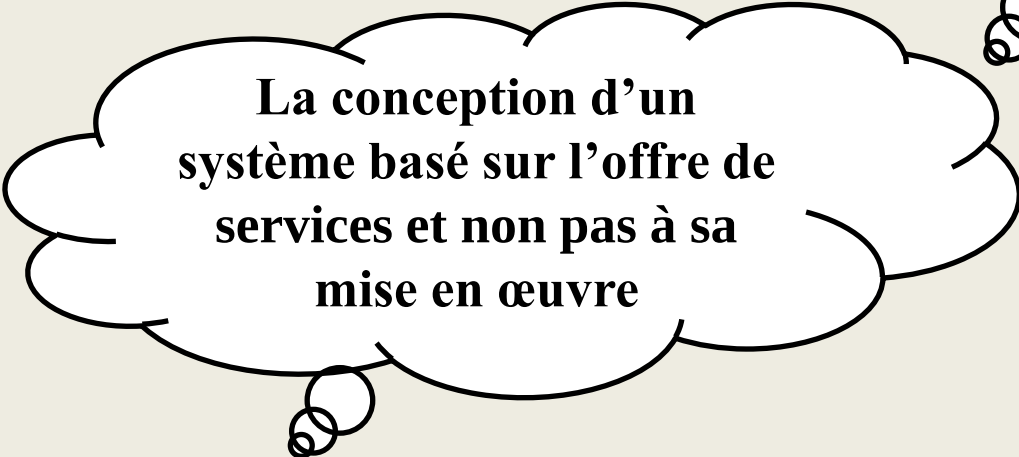
#

Modéliser le système informatique en se basant sur la base de données et les IHMs

Définitions:



Un style architectural qui met l'accent sur la mise en œuvre des composants



La conception d'un système basé sur l'offre de services et non pas à sa mise en œuvre

SOA ?

Problématique de l'intégration: (1)

- Les entreprises doivent s'adapter en permanence et être réactive aux variations des marchés.
- Ces changements ont un impact sur le système d'information (SI) des entreprises.
- L'intégration difficile des SI = frein à ces changements.



L'activité de l'entreprise qui pilote la technologie et non pas l'inverse

Problématique de l'intégration: (2)

- Souvent le développement des applications est piloté par les besoins de l'entreprise.
- La modélisation et le développement sont dirigé par les contraintes technologiques et financières.



Le décalage entre les besoins métier de l'entreprise et la réalisation.

L'évolution des besoins fonctionnels de l'application est ignorée

Problématique de l'intégration: (3)

- Le découpage Présentation – Traitement – Données ou bien l'architecture 3-tiers facilite les tâches du MOE (Maitre d'oeuvre).



Absence de mutualisation de développement entre les projets chose qui peut donner des redondances au niveau des fonctions.

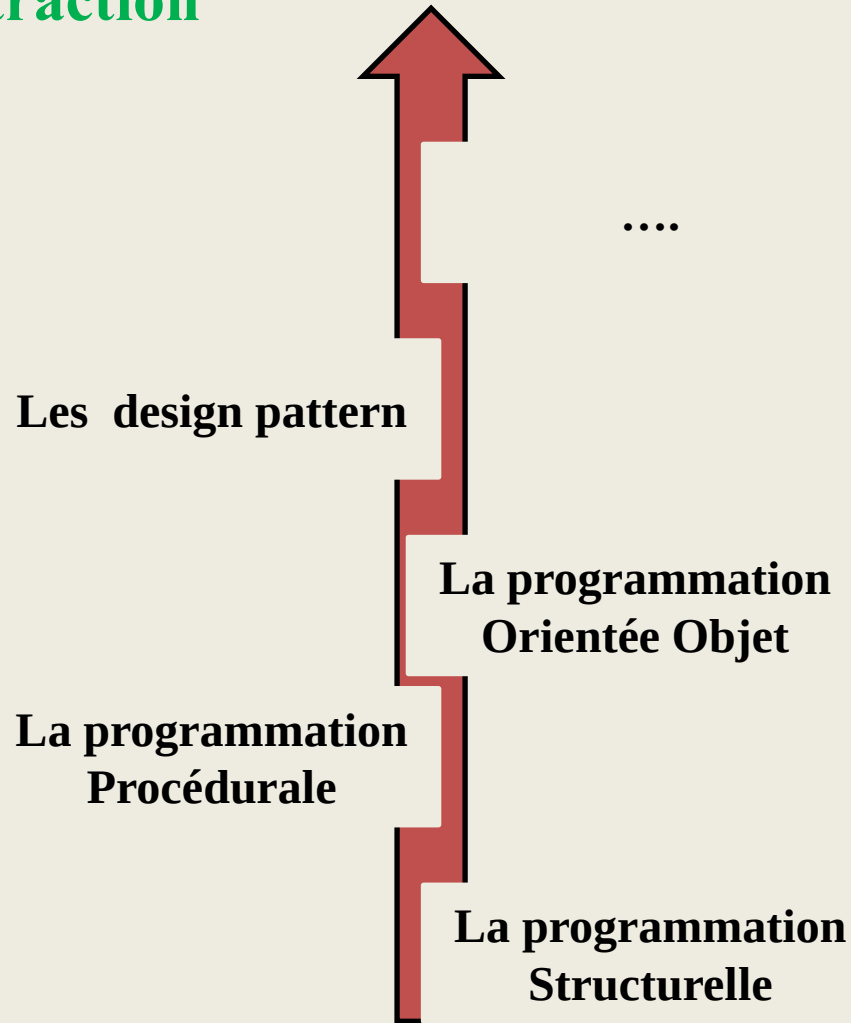
Problématique de l'intégration: (4)

- L'organisation de l'entreprise en départements fonctionnels
- Les processus métiers de l'entreprise demande le transfert d'un flux d'informations important de données entre les départements de l'entreprise d'un côté et entre l'entreprise et les partenaires extérieurs.



Coûts considérables dans la gestion des flux d'informations

Vers plus d'abstraction



Limites de la POO

- **Couplage fort**
- **Interactions entre les objets sont cachées dans le code.**
- **Structure et architecture de l'application est peu visibles.**



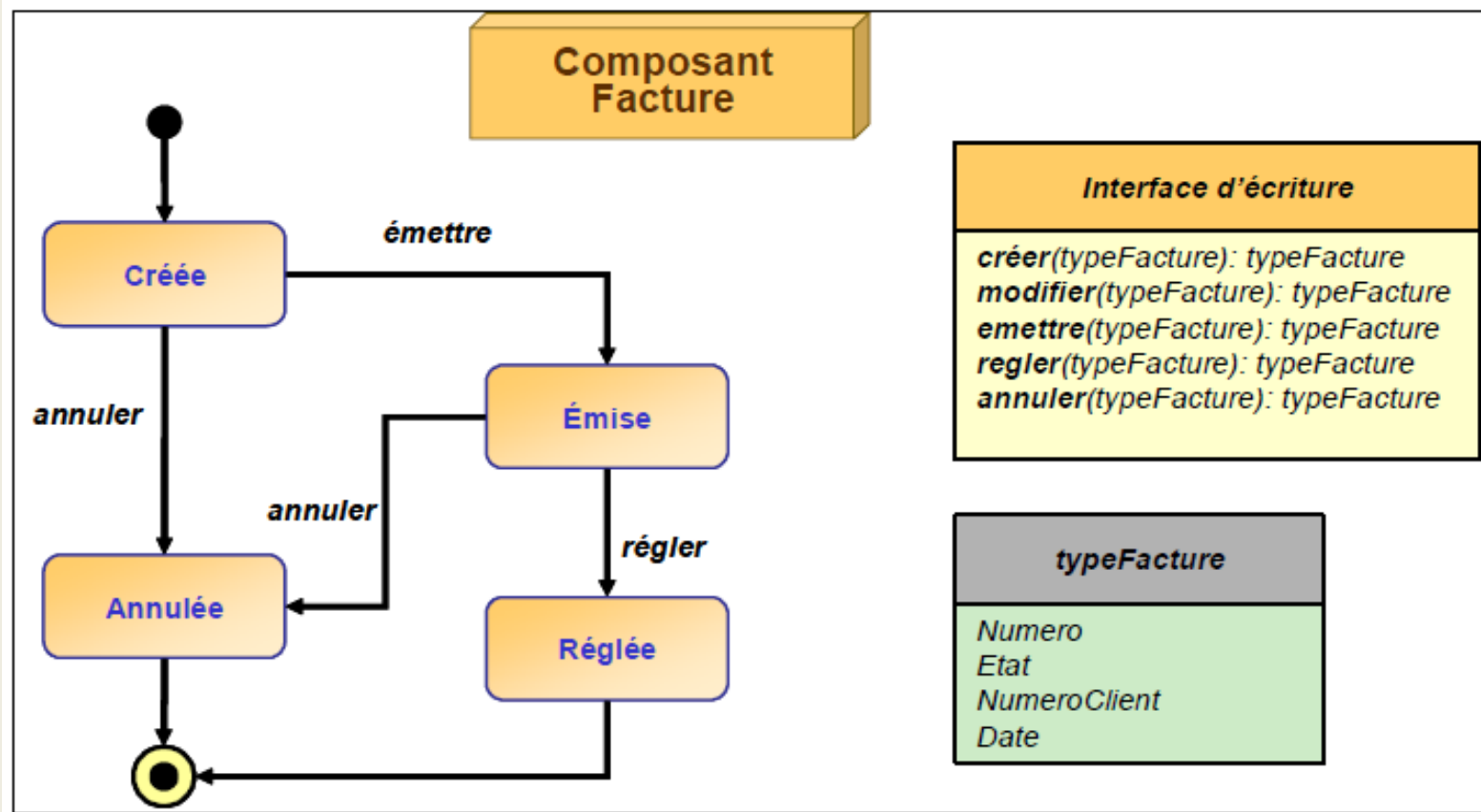
rend difficile la réutilisation du code, augmente la complexité du système et bloque l'évolution des systèmes.

Vers le composant de service

- **Une unité regroupant les fonctionnalités concernant une même idée.**
- **La brique de base de l'architecture SOA.**
- **Des unités autonomes, configurables, qui exporte des attributs et des méthodes.**
- **Les interactions entre les composantes sont définies par: ce qui est fourni par le composant, ce qui est utilisé par le composant ainsi les modes de communication.**

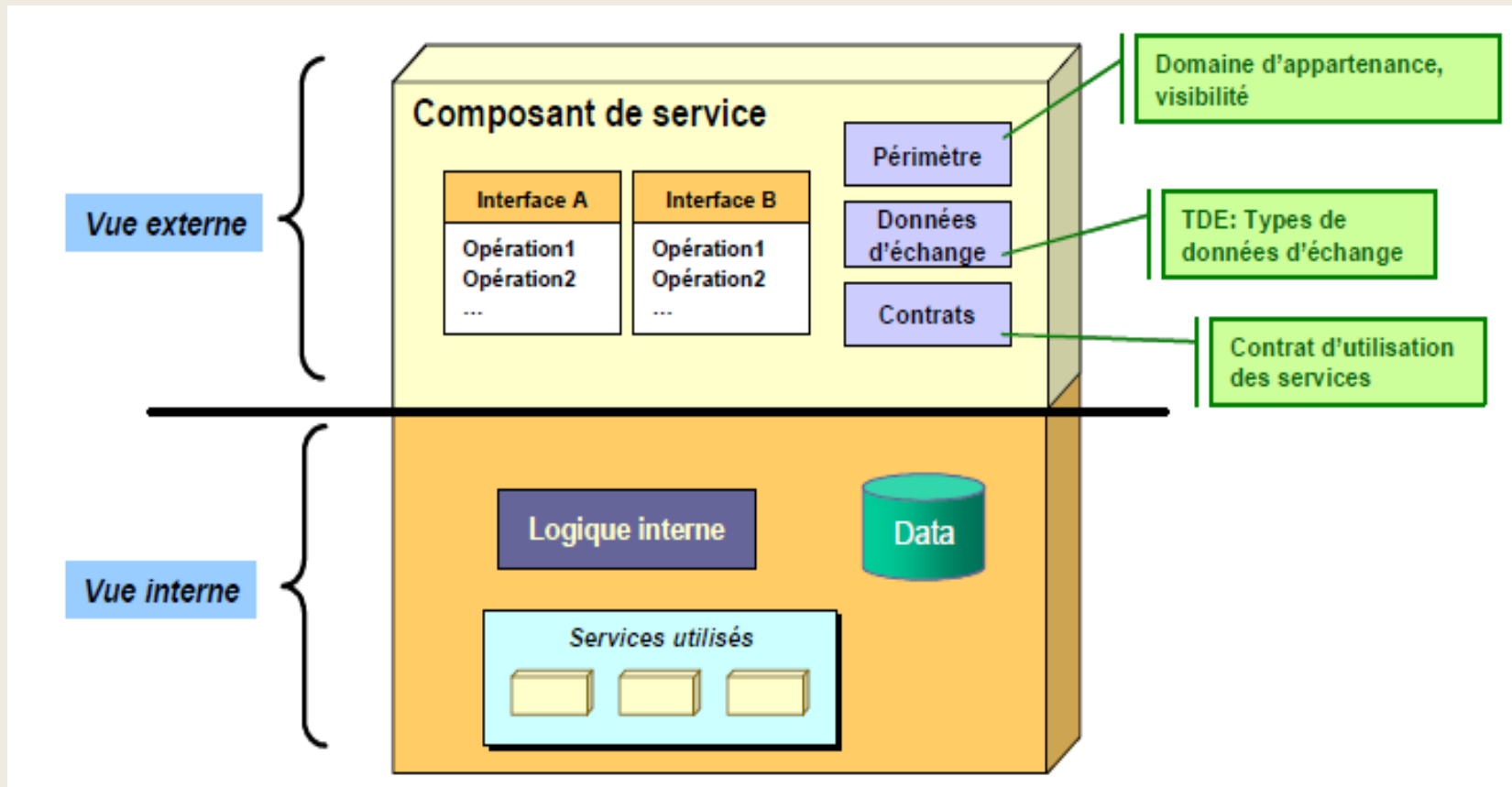
Vers le composant de service

Exemple du composant de service:

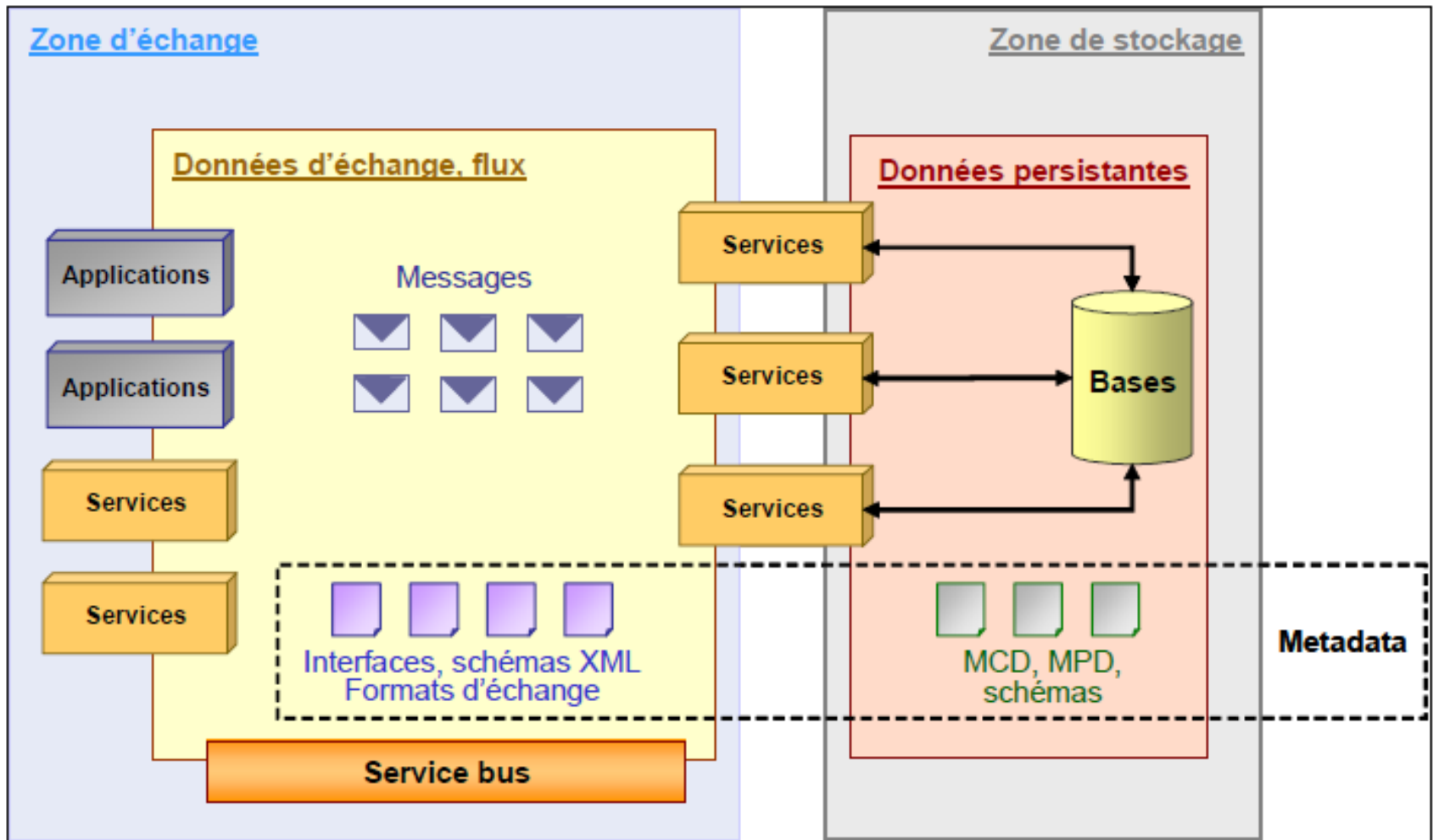


Vers le composant de service

Structure du composant de service:

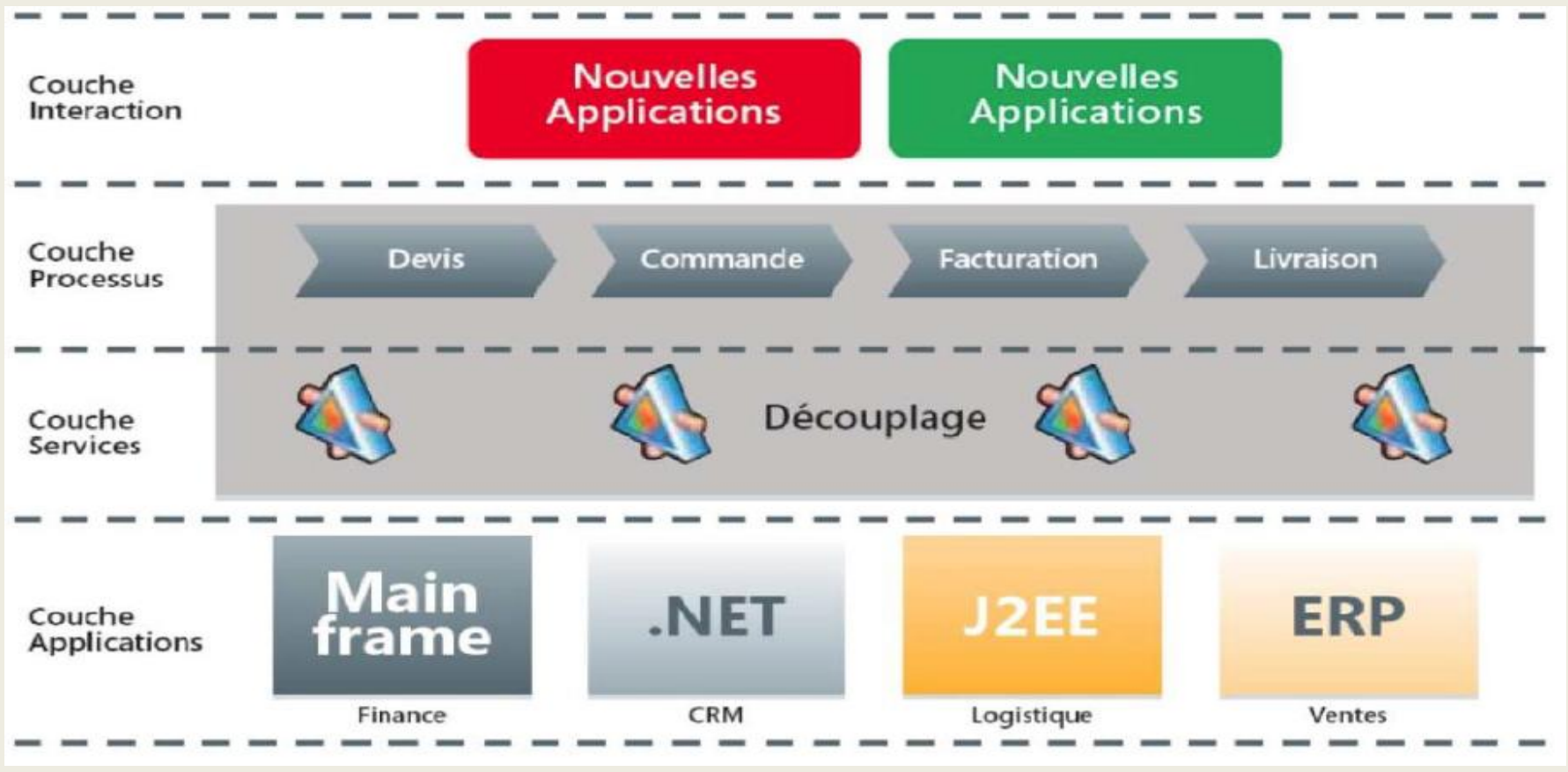


Données d'échanges et données persistantes



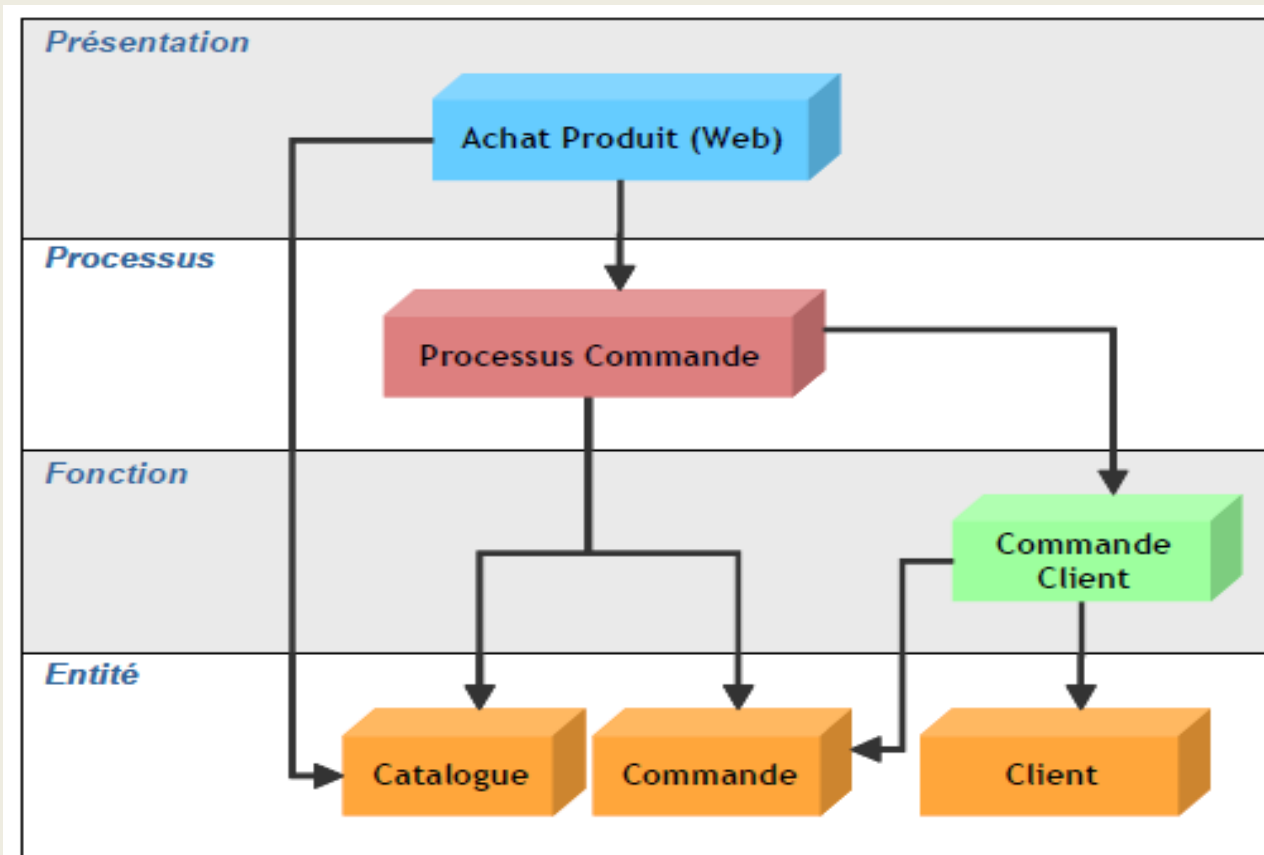
Principes fondamentaux de l'architecture SOA:

1- Architecture de base SOA



Principes fondamentaux de l'architecture SOA:

1- Architecture de base SOA: Exemple



Principes fondamentaux de l'architecture SOA:

2- L'orientation service, Service au sens SOA ?

« Un service au sens SOA, met à disposition d'acteurs intervenants dans des processus métier un accès vers une ou plusieurs fonctions métiers ». [Fournier-Morel, Grojean, Plouin, Rognon, 2011, p. 30].

- Propriétés du service :

Un service expose un contrat:



Principes fondamentaux de l'architecture SOA:

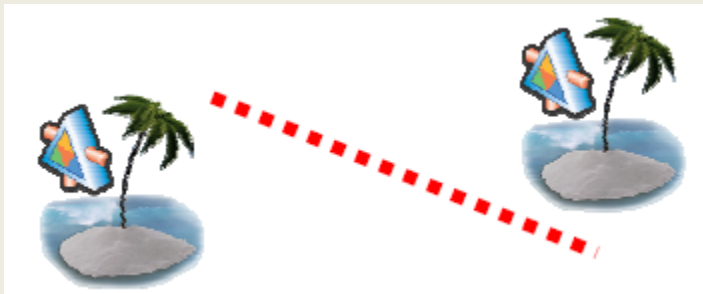
2- L'orientation service, Service au sens SOA ?

- Propriétés du service :

Les services communiquants par messages :



Les services sont faiblement couplés :



Principes fondamentaux de l'architecture SOA:

2- L'orientation service, Service au sens SOA ?

- **Propriétés du service :**

un service est autonome et capable de communiquer avec les autres services :



Autonome = capable de gérer ses ressources sans intervention externe

Principes fondamentaux de l'architecture SOA:

2- L'orientation service, Service au sens SOA ?

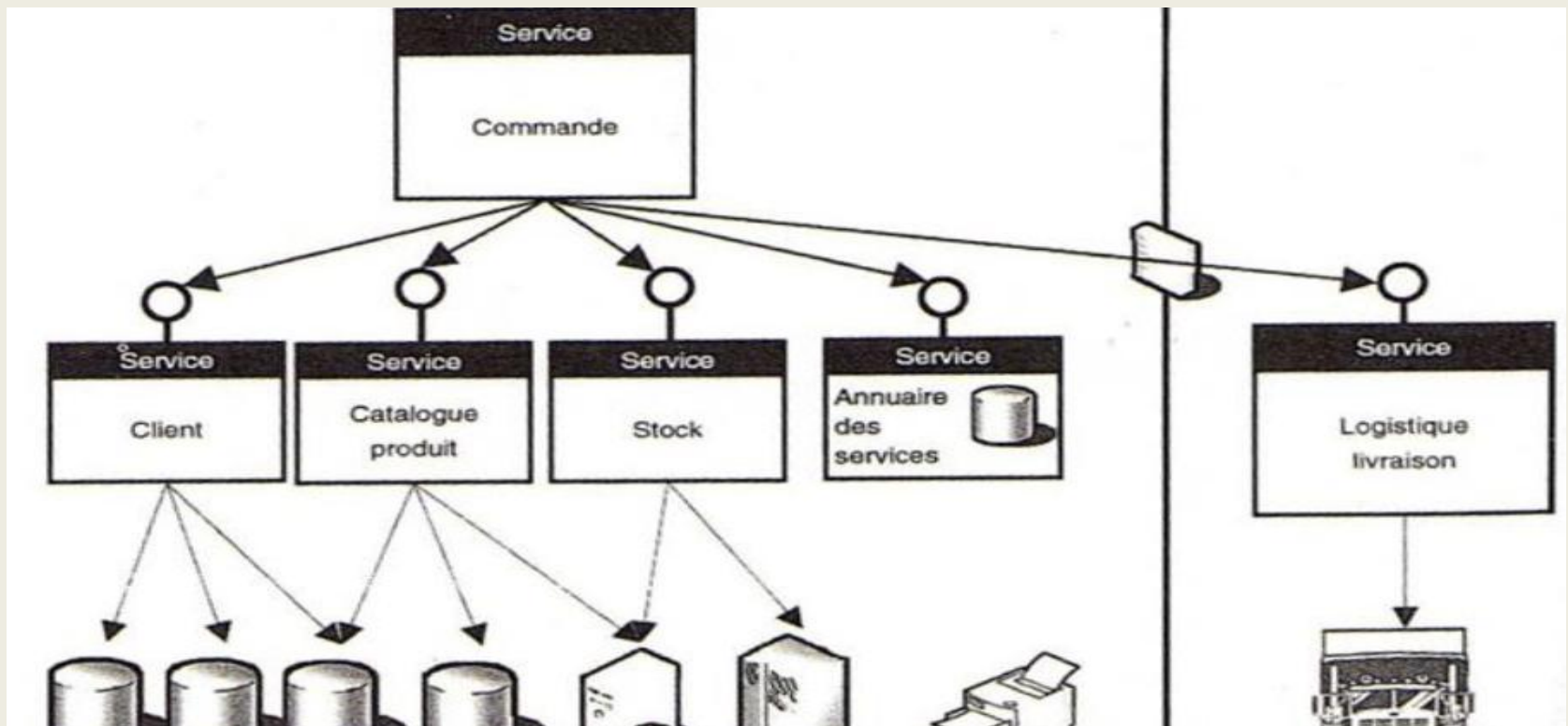
- **Composition des services:** Les services peuvent être individuellement utiles, ou ils peuvent être intégrés, composé pour fournir des services de haut niveau. Ce qui favorise la réutilisation des fonctionnalités existantes. Cette notion est définie comme « **la composition de services** ».

La composition de services = La réutilisation par composition

Principes fondamentaux de l'architecture SOA:

2- L'orientation service, Service au sens SOA ?

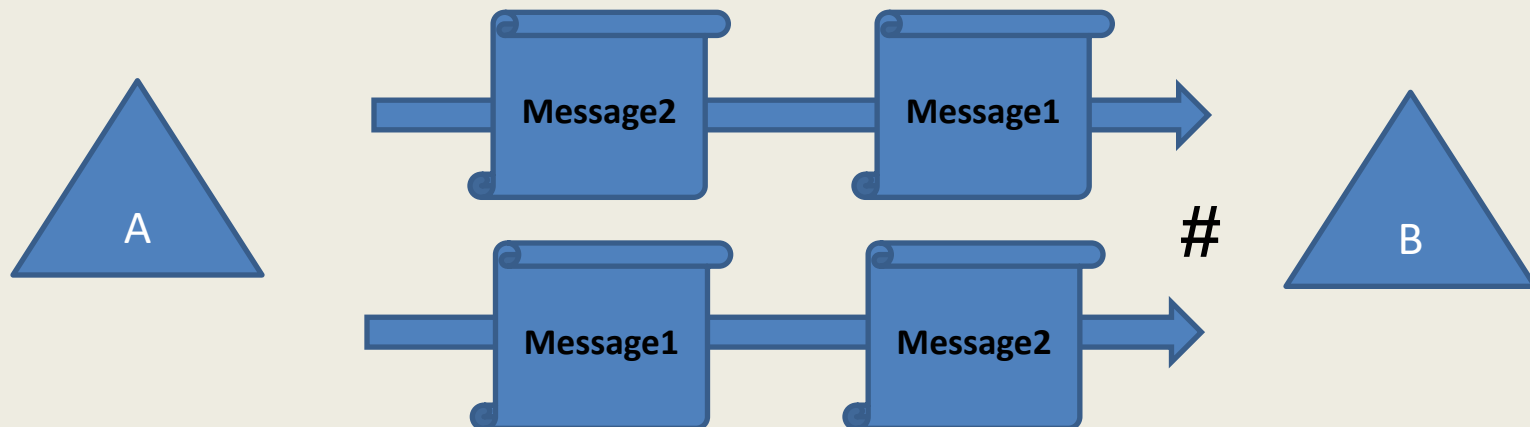
Exemple de composition de services:



Principes fondamentaux de l'architecture SOA:

2- L'orientation service, Service au sens SOA ?

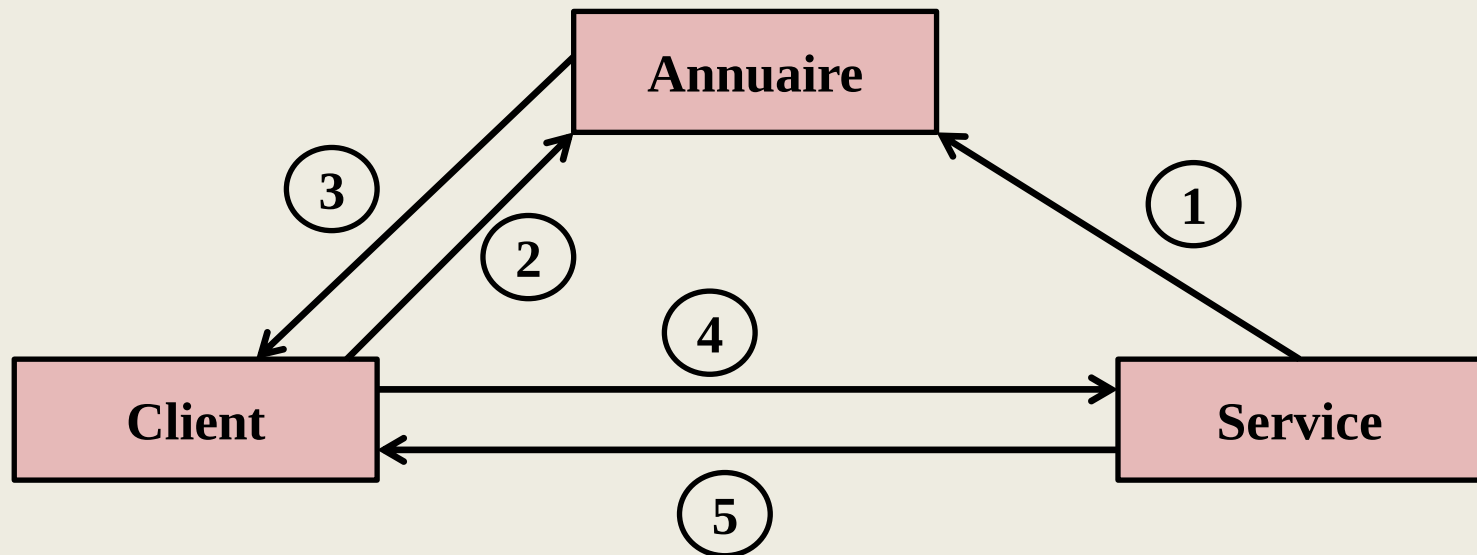
- **La chorégraphie de services: c'est l'ordre d'appel des services; les services peuvent participer à un workflow, où l'ordre dans lequel les messages sont envoyés et reçus affecte le résultat des opérations effectuées par un service.**



Principes fondamentaux de l'architecture SOA:

2- L'orientation service, Service au sens SOA ?

- Cycle d'interaction des services:



Principes fondamentaux de l'architecture SOA:

2- L'orientation service, Service au sens SOA ?

- **Service avec état = Un service qui sauvegarde son dernier état à fin d'en disposer en cas de besoin.**

Exemples:

- **Services de réservation en ligne d'un billet d'avion et le paiement en ligne. Composé de deux services qui s'exécutent en succession.**
- **Services de consultation et achat des actions en bourse en ligne**

Principes fondamentaux de l'architecture SOA:

2- L'orientation service, Service au sens SOA ?

- **Service sans état = Un service qui ne sauvegarde pas son dernier état.**

Exemples:

- **Dans ce cas, il est facile de voir qu'une fois un client a demandé et reçu des informations, l'opération est terminée, et le client n'a pas besoin particulier de revoir le même service pour ses besoins futurs.**

Principes fondamentaux de l'architecture SOA:

2- L'orientation service, Service au sens SOA ?

Solution possible: Pour une transaction plus complexe qui nécessite plusieurs étapes, la conception du service pourrait être tel que le service conserve dans sa mémoire locale de l'information – état - au sujet de la première étape, s'attendant à en faire usage lorsque les clients les contactant pour la prochaine étape.

Limites: Dans ce cas, le service est «avec état», et le client utilise cet état pour la prochaine étape. Cela pourrait entraîner un retard si de nombreux clients utilisent le même service ou un échec de la transaction si le nœud qui héberge le service échoue entre les étapes.

Principes fondamentaux de l'architecture SOA:

2- L'orientation service, Service au sens SOA ?

Encore une solution: Sauvegarde de l'état chez le client

- À la fin de chaque étape intermédiaire le service doit restituer au client les informations suffisantes sur l'état pour permettre à tout service qualifié d'identifier et de poursuivre la transaction.
- Le client doit récupérer les informations d'état du service qu'il choisit pour traiter la prochaine étape de la transaction.
- Le service choisi doit être en mesure d'accepter et de traiter les informations d'état fournies par le client, même si ce n'est pas le même service qui a traité les étapes précédentes.

Principes fondamentaux de l'architecture SOA:

2- L'orientation service, Service au sens SOA ?

- Le cycle de vie d'un service



Principes fondamentaux de l'architecture SOA:

3- Couplage faible ou lâche

- **Deuxième principe essentiel de la SOA.**
- **Le couplage est une métrique, d'intensité forte ou faible.**
- **Indiquant le niveau d'interaction entre deux composants de services.**
- **Correspond au degré d'échange d'informations entre les composants service du système.**
- **Le couplage faible offre plusieurs avantages:**

Principes fondamentaux de l'architecture SOA:

3- Couplage faible ou lâche

Avantage	Description
Flexibilité	Un service peut être situé sur n'importe quel serveur, et déplacées au besoin. Tant qu'il maintient son entrée de registre, les clients potentiels seront en mesure de le trouver.
Évolutivité	Les services peuvent être ajoutés et supprimés, car la demande des clients varie. (supprimer le service inutile)

Principes fondamentaux de l'architecture SOA:

3- Couplage faible ou lâche

Avantage	Description
Interchangeabilité	la mise en ligne de nouvelles versions des services est transparente pour les utilisateurs. Les mises à jour des services se font sans interruption pour les utilisateurs.
Maintenance et la tolérance aux pannes	Si un serveur, un composant logiciel, ou un segment de réseau tombe en panne, ou le service devient indisponible pour toute autre raison, les clients peuvent interroger le Registre pour les services de remplacement qui offrent les fonctionnalités requises, et de continuer à fonctionner sans interruption.

- **La mise en œuvre d'une SOA implique:**
 1. Une **démarche d'identification des services** (Metier et Applicatifs) à exposer.
 2. La définition et l'implementation d'un **socle technique** supportant ces services .

- **La démarche SOA doit être progressive selon:**
 1. Approche Top/Down: Périmètre métier par Périmètre métier.
 2. Approche Bottom/Up : application par application.

- **La mise en œuvre d'une SOA implique:**
 1. **Approche Top/Down: Périmètre métier par Périmètre métier.**
 - L'objectif est d'identifier les besoins fonctionnels à couvrir avec des services.
 - Cartographie des processus métier : Modélisation des workflows et des activités principales.
 - Définition des cas d'usage : Identifier les interactions des utilisateurs avec le système.
 - Identification des services métier : Définir les services candidats en fonction des besoins métier.
 - Exemple : Pour un SI d'une banque, un processus métier peut être « Ouverture d'un compte", qui implique plusieurs sous-processus (vérification de l'identité, approbation, délivrance d'un RIB, ...).

- **La mise en œuvre d'une SOA implique:**
 1. **Approche Bottom/Up : application par application.**
 - On analyse les ressources et applications déjà en place pour éviter les redondances.
 - Recensement des applications et bases de données existantes.
 - Identification des services réutilisables (ex. API REST, Web Services SOAP, microservices existants).
 - Détection des contraintes techniques (performances, sécurité, intégration).
 - Exemple : Un service de vérification de l'identité peut déjà exister sous forme d'une API dans un système externe.

- **Démarche d'identification des services SOA selon une approche **Meet in the Middle**:**
- L'objectif est d'harmoniser les services métier avec les capacités techniques disponibles.
- Élimination des doublons entre les services identifiés dans les deux approches.
- Découpage et normalisation des services pour respecter les principes SOA (réutilisabilité, couplage faible).
- Définition des niveaux de service :
 - ✓ Services métier : alignés avec les processus métier.
 - ✓ Services applicatifs : exposant des fonctionnalités spécifiques.
 - ✓ Services d'intégration : permettant la communication entre les systèmes existants.
- Exemple : Plutôt que de créer un nouveau service de vérification de l'identité, on expose l'API existante sous forme d'un service métier standardisé utilisable par d'autres processus.

- **Démarche d'identification des services SOA selon une approche **Meet in the Middle**:**

Étape 1: Formalisation des processus fonctionnels.

Étape 2: Décliner le processus sur les domaines fonctionnels du SI.

Étape 3-a: Identification des services existants répondant aux fonctionnalités.

Étape 3-b: Identifier les services composites.

Étape 3-c: Déterminer les nouveaux services.

Étape 4: Associer les services aux activités.

Étape 5: Enrichir les processus avec les cas alternatifs.

- **Démarche d'identification des services SOA selon une approche **Meet in the Middle**:**

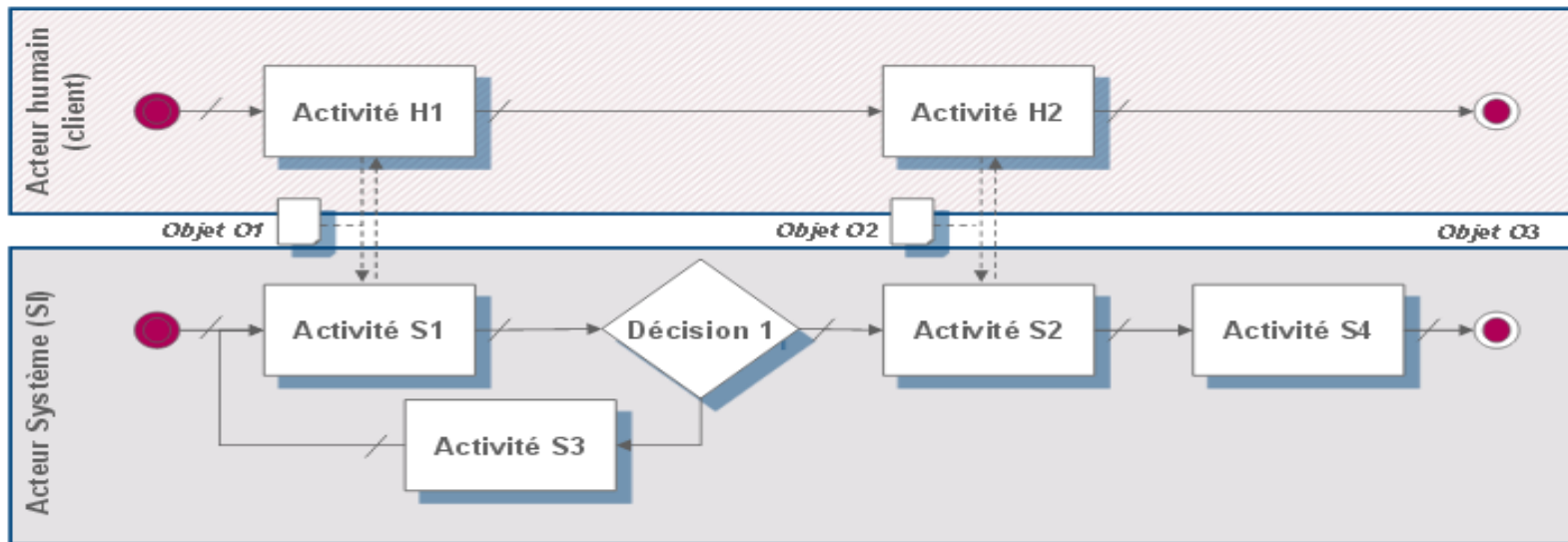
Étape 1: Formalisation des processus fonctionnels.

- ☐ Un processus = Sous processus + décisions (Business rules) + activités.
- ☐ Chaque processus possède des entrées et des sorties.
- ☐ Un processus est l'orchestration de services.
- ☐ Les processus fonctionnels sont réalisés par le MOA (Maitre d'ouvrage).
- ☐ BPM (Business Process Management): Donne à l'Entreprise les moyens de gérer ses processus métiers de manière informatisée (modélisation, simulation, execution et audit)

- Démarche d'identification des services SOA selon une approche **Meet in the Middle**:

Étape 1: Formalisation des processus fonctionnels.

- ❑ Une activité ne doit pas manipuler plus qu'un objet métier (La granularité).
- ❑ Une activité ne doit pas avoir des fonctions du plan applicatif.



22 novembre 2013 - Propriété de Solucom, reproduction interdite

- **Démarche d'identification des services SOA selon une approche **Meet in the Middle**:**

Étape 2: Décliner le processus sur les domaines fonctionnels du SI.

- ☐ permet principalement d'identifier les domaines du SI impactés par le processus fonctionnel.
- ☐ vérifier que le processus implémente bien la logique d'isolement fonctionnel au niveau du SI.
- ☐ Cette étape utilise principalement le MOM et le POS.
- ☐ MOM = Modèle d'Objet Métier.
- ☐ POS = Plan d'Occupation des Sols du SI.

- **Démarche d'identification des services SOA selon une approche **Meet in the Middle**:**

Étape 2: Décliner le processus sur les domaines fonctionnels du SI.

- ❑ **MOM - Modèle d'Objet Métier:** est une représentation des entités métier, de leurs attributs, et de leurs relations dans un système. Il sert à structurer les données et à modéliser les interactions entre différents concepts métier.
- ❑ **POS - Plan d'Occupation des Sols du SI:** une cartographie des applications, données et flux au sein d'un Système d'Information (SI). Il permet d'avoir une vue globale des composants, de leurs interactions et de détecter les redondances ou incohérences; rôles:
 - ✓ Cartographier les applications et leurs rôles.
 - ✓ Identifier les échanges de données entre les systèmes.
 - ✓ Optimiser le SI en éliminant les doublons et en améliorant les interactions.

- **Démarche d'identification des services SOA selon une approche **Meet in the Middle**:**

Étape 3-a: Identification des services existants répondant aux fonctionnalités.

Étape 3-b: Identifier les services composites.

Étape 3-c: Déterminer les nouveaux services.

Étape 4: Associer les services aux activités.

Étape 5: Enrichir les processus avec les cas alternatifs.

- **Démarche d'identification des services SOA selon une approche **Meet in the Middle**:**

Une fois les services identifiés et modélisés :

- ✓ Prototypage et validation avec les métiers et les équipes IT.
- ✓ Définition des contrats de service (WSDL, OpenAPI, JSON, XML, etc.).
- ✓ Déploiement des services dans un ESB (Enterprise Service Bus) ou via des API Gateway.

Exemple : Un service **d'évaluation de crédit** est conçu en combinant plusieurs services existants (vérification de score, consultation des antécédents bancaires, analyse des risques, ...).



Partie 3

SoaML

Service oriented architecture Modeling Language (SoaML)

- Extensions à UML2 pour supporter les concepts de service.
 - Se concentre sur les concepts fondamentaux de modélisation des services.
 - Une base pour de futures extensions et une intégration avec BPMN, BMM et d'autres métamodèles.
- ✓ **BPMN (Business Process Model and Notation)** est un standard graphique utilisé pour modéliser les processus métier d'une organisation. Il permet de représenter visuellement les étapes d'un processus, les acteurs impliqués et les flux d'informations.
 - ✓ **Le BMM (Business Motivation Model)** est un cadre conceptuel qui permet de modéliser la motivation stratégique d'une organisation. Il définit les objectifs, les stratégies et les influences externes qui guident les décisions de l'entreprise.

SoaML: historique

- **Années 2000 : L'essor des SOA**
- **2006 : Début du développement de SoaML**
 - ✓ **L'OMG - Object Management Group, lance un appel à propositions pour une norme SOA basée sur UML.**
 - ✓ **Plusieurs entreprises et organismes (IBM, Model Driven Solutions, Adaptive, etc.) collaborent pour définir une méthodologie de modélisation des services.**
- **2009 : Standardisation de SoaML par l'OMG**
- **Depuis 2010 : Adoption et évolutions**

SoaML: Objectifs

- **Standardiser la Modélisation des Services,**
- **Aligner les Services avec les Besoins Métier,**
- **Faciliter la Conception et l'Implémentation de SOA,**
- **Assurer une Interopérabilité entre les Composants du SI**
- **Favorise l'implémentation du MDA – Model Driven Architecture,**

SoaML: MDA → Model-Driven Architecture

- **Une approche standardisée par l'OMG (Object Management Group) qui vise à séparer la logique métier d'un système des détails techniques de sa mise en œuvre. Elle repose sur l'utilisation intensive de modèles pour guider le développement logiciel.**
 - ✓ **Séparer les préoccupations métier et techniques pour faciliter l'évolution des systèmes.**
 - ✓ **Automatiser la transformation des modèles en code source via des outils de génération.**
 - ✓ **Améliorer la portabilité en rendant le modèle indépendant des plateformes technologiques**

SoaML: MDA → Model-Driven Architecture

CIM (Computation Independent Model)

Modèle métier, sans considération technique.

- Diagrammes BPMN décrivant les processus d'entreprise.

PIM (Platform Independent Model)

Modèle indépendant des technologies, décrivant la logique métier.

- Diagrammes UML représentant les classes et interactions.
- Diagrammes SoaML.
- Transfert des données XML, JSON, ...

PSM (Platform Specific Model)

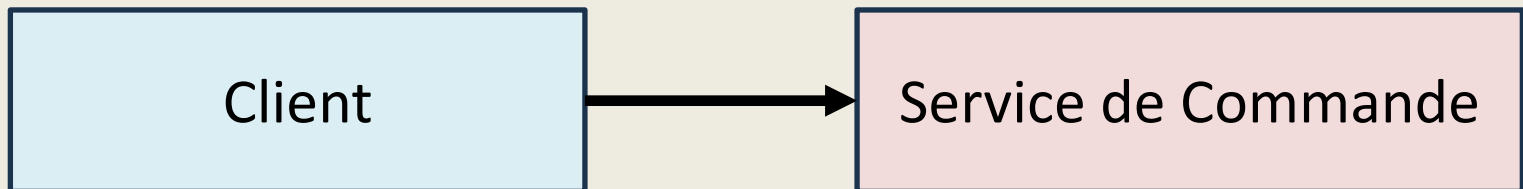
Modèle spécifique à une plateforme (Java, .NET, Cloud, etc.).

- Implémentation en Java avec des frameworks comme Spring.

SoaML: Diagrammes

1. **Diagramme de Participants (Participant Diagram):** Il représente les acteurs (fournisseurs et consommateurs de services) et leurs interactions.

Exemple: Système de gestion de commande

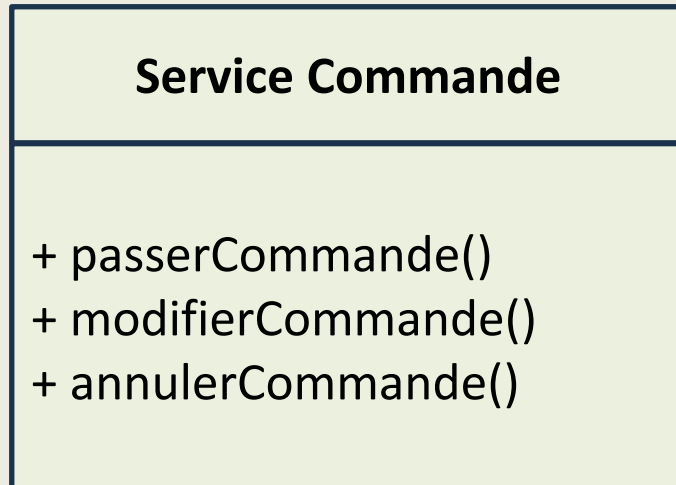


Peut être représenté par un diagramme de classes avec des interfaces de services.

SoaML: Diagrammes

2. **Diagramme de Port et Interfaces (Service Interface Diagram):** Ce diagramme décrit les **services exposés** par un participant et les interactions possibles.

Exemple : Interface du Service de Commande



- **ServiceCommande** : Port du fournisseur
- **InterfaceCommande** : Définit les opérations
du service

SoaML: Diagrammes

- 3. Diagramme de Contrat de Service (Service Contract Diagram):** Il définit un contrat de service entre plusieurs participants et décrit les responsabilités de chaque partie.

Exemple : Contrat entre un Client et un Service de Livraison

Contrat Livraison

Rôles : Client, Transporteur

Obligations :

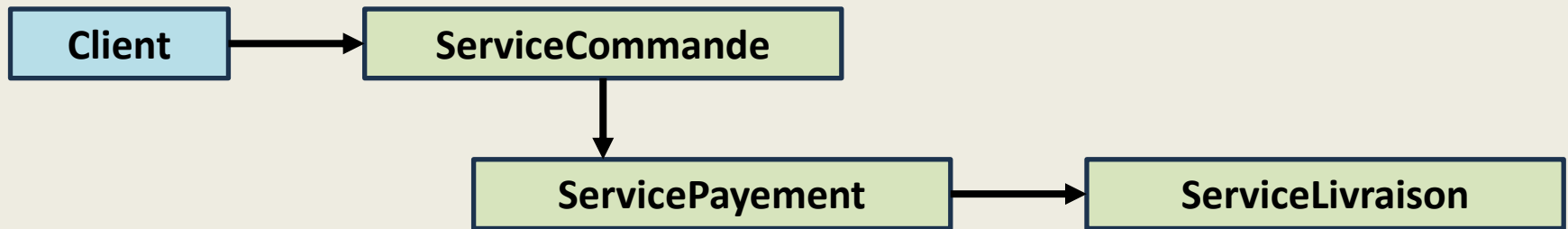
- Le client fournit l'adresse
- Le transporteur livre en 24h

- Client demande la livraison
- Transporteur assure la livraison

SoaML: Diagrammes

4. **Diagramme de Composition de Service (Service Composition Diagram):** Il montre comment plusieurs services interagissent pour former un service plus complexe.

Exemple : Gestion d'une Commande en ligne



Correspond à un diagramme de séquence montrant l'enchaînement des services.